A

MAJOR PROJECT REPORT

ON

FILE UPLOADING IN CUSTOMER RELATIONSHIP MANAGEMENT

Submitted in partial fulfilment of the requirements for award of the degree of

BACHELOR OF TECHNOLOGY

In

Computer Science Engineering

BY

A. NARESH	**237W5A0501**
A. AKASH	**237W5A0502**
A. JAYANTH	**237W5A0503**

Under the guidance of

Ms. Rajeshwari Vanam, M Tech

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

St. Mary's Integrated Campus

(Approved by AICTE, NEW DELHI. & Affiliated to JNTU-HYDERABAD, Accredited by NAAC)

Deshmukhi (V), Pochampally (M), Yadadri Bhuvanagiri (D), Telangana -508284

[2025-26]

CERTIFICATE

This is to certify that project report entitled **"FILE UPLOADING IN CUSTOMER RELATIONSHIP MANAGEMENT"** is bonafide work carried out in the IV/II semester by **A. Naresh 237W5A0501, A. Akash 237W5A0502, A. Jayanth 237W5A0503** in partial fulfilment award of Bachelor of Technology in **CSE** from **St. Mary's Integrated campus** during the academic year 2023-26.

INTERNAL GUIDE	HEAD OF THE DEPARTMENT
Ms. Rajeshwari Vanam, M Tech	**Dr. R. Srikanth Reddy, M Tech, PhD**
Dept. of CSE	**Dept. of CSE**

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of this project would be incomplete without the mention of the people who made it possible. We consider it as a privilege to express our gratitude and respect to all those who guided us in the completion of the project.

We are thankful to our internal guide **Ms. Rajeshwari Vanam, M. Tech**, in **Department of CSE, St. Mary's Integrated Campus** for having been of a source encouragement and for insisting to do this project work.

We are obliged to **Mr. R. SRIKANTH REDDY, Head of the Department of CSE, St. Mary's Integrated Campus** for his guidance and suggestion throughout project work.

We take this opportunity to express a deep sense of gratitude to **Dr. E. SWATHIN GOUD, Principal of St. Mary's Integrated Campus** for allowing us to do this project and for this affectionate encouragement in presenting this project work.

We convey our sincere thanks to **Sri Rev. K.V.K RAO, Chairman** of **St. Mary's Integrated Campus** for giving us learning environment to grow out self personally as well as professionally.

We would like to express our thanks to all staff members who have helped us directly and indirectly in accomplishing this project work. We also extended our sincere thanks to our parents and friends for their moral support throughout the project work. Above all we thank god almighty for his manifold mercies in carrying out this project work successfully.

A. Naresh	**237W5A0501**
A. Jayanth	**237W5A0503**
A. Akash	**237W5A0502**

DECLARATION

This is to certify that the work report in thesis titled, "**FILE UPLOADING IN CUSTOMER RELATIONSHIP MANAGEMENT**", submitted to **the Department of CSE**, **St. Mary's Integrated Campus** in fulfilment of degree for the award of Bachelor of Technology, is a bonafide work done by us. No part of the thesis is copied from books, journals or internet and wherever the portion is taken, the same has been duly referred in the text. The reported results are based on the project work entirely done by us and not copied from any other sources. Also we declare that the matter embedded in this thesis has not been submitted by us in full or partially there for the award of any degree of any other institution or university previously

A. Naresh	**237W5A0501**
A. Jayanth	**237W5A0503**
A. Akash	**237W5A0502**

ABSTRACT

The **Secure File Management System** is a comprehensive full-stack platform designed to provide a secure environment for the upload, retrieval, and management of digital files. Built using **Spring Boot 3.2** and **React 18**, the system addresses modern security requirements by implementing stateless **JSON Web Token (JWT)** authentication and a robust **Role-Based Access Control (RBAC)** framework.

The technical architecture follows a standard three-layer model, utilizing **Java 17** and **Spring Security 6.x** to enforce strict access boundaries. Users are categorized into **ADMIN** and **CUSTOMER** roles; administrative users possess global visibility into system statistics and user data, while customers are restricted to managing their own files within a private context. The system employs **MySQL** for structured metadata persistence and a dedicated server filesystem for binary storage, ensuring an efficient separation of data.

Key technical features include centralized **Axios interceptors** for automated session handling, secure password hashing via **BCrypt**, and a seamless file CRUD (Create, Read, Update, Delete) pipeline. This project demonstrates a production-ready approach to secure cloud-oriented file management, emphasizing scalability, data integrity, and delegated authorization.

CONTENTS

S.NO.	CHAPTER NAME	PAGE NO
1.	**INTRODUCTION**	1
2.	**LITERATURE SURVEY**	3
3.	**SYSTEM ANALYSIS**	4
	3.1. Existing System	
	3.2. Proposed System	
4.	**SYSTEM STUDY**	7
	4.1 Economic Feasibility	
	4.2. Technical Feasibility	
	4.3. Social Feasibility	
	4.4. Operational Feasibility	
5.	**TECHNOLOGY STACK**	11
6.	**SYSTEM SPECIFICATIONS**	12
	6.1. Software Requirements	
	6.2. Hardware Requirements	
7.	**SYSTEM DESIGN**	14
	7.1. System Architecture	
	7.2. CRM Diagrams	
8.	**IMPLEMENTATION / MODULES**	31
9.	**SOFTWARE ENVIRONMENT**	38
10.	**SOURCE CODE**	51
11.	**SYSTEM TESTING**	59
12.	**OUTPUT SCREENS**	61
13.	**CONCLUSION**	65
14.	**FUTURE ENHANCEMENT**	66

LIST OF FIGURES

S.NO	FIGURE NO	FIGURE NAME	PAGE NO
1.	7.1	Master Architecture	16
2.	7.2	System Architecture Flow	17
3.	7.3	Architecture Detailed View	18
4.	7.4	Data Flow Diagram	19
5.	7.5	Use Case Diagram	21
6.	7.6	Class Diagram	22
7	7.7	Sequence Diagram	24
8.	7.8	Activity Diagram	25
4.	8.1	Authentication Flow	32
5	8.2	File Upload Flow	37
6.	9.1	User Interface Login	40
7.	9.2	User Interface Register	41
8.	9.3	User Interface Root	42
9.	9.4	User Interface Login Screen	43
10.	9.5	User Interface Register Screen	44
11.	9.6	User Interface Dashboard Screen	45
12.	9.7	Backend Setup Flow	46
13.	9.8	Frontend Backend Integration	50
14	12	Output Screens	61

CHAPTER - 1
INTRODUCTION

In modern Customer Relationship Management (CRM) systems, the ability to securely store and manage documents is as critical as managing customer data itself. This project, a Secure File Management System, is designed to function as a core module for such CRM platforms. It provides a full-stack solution for authenticated users to upload, list, and download files within a secure environment. By utilizing a high-performance stack comprising Spring Boot 3.2 and React 18, the system ensures that sensitive customer documents are protected through stateless JWT authentication and strict Role-Based Access Control (RBAC) Core Purpose and Functionality

The system's primary objective is to offer a secure, role-based platform where users can interact with their data under strict authorization boundaries.

- **Secure File Handling**: Users can perform standard CRUD operations—uploading, listing, downloading, and deleting files—within an authenticated context.
- **Role-Based Access Control (RBAC)**: The application differentiates between **ADMIN** and **CUSTOMER** roles to ensure data isolation and administrative oversight.
- **Identity Management**: Security is enforced through stateless **JWT (JSON Web Token)** authentication, allowing for secure login and registration flows.

Technical Foundation

The project leverages industry-standard technologies to ensure scalability and security:

- **Backend**: Powered by **Spring Boot 3.2** and **Java 17**, utilizing **Spring Security 6.x** for complex authentication and authorization filters.
- **Frontend**: Developed with **React 18** and **TypeScript**, providing a type-safe, responsive user interface styled with **Tailwind CSS** and **shadcn/ui**.
- **Data Management**: **MySQL 8.0** serves as the relational database for storing user profiles and file metadata, while the server filesystem handles binary file persistence.

Architectural Approach

Following a **three-layer architecture** (Presentation, Application, and Data), the system maintains a clean separation of concerns.

- The **Presentation Layer** manages role-aware navigation and protected client-side routes.
- The **Application Layer** processes business logic through REST controllers and specialized service layers.
- The **Data Layer** ensures data integrity and reliable file storage.

Objectives of the System:

The primary goal of this system is to bridge the gap between user accessibility and data security within a CRM framework. The specific objectives include:

- **Secure Authentication**: Implementing a robust login and registration system where user sessions are managed via **JSON Web Tokens (JWT)** to ensure stateless and secure API interactions.

- **Hierarchical Access Control**: Establishing **ADMIN** and **CUSTOMER** roles to ensure that while administrators have global visibility, standard customers can only access their own uploaded documents.

- **Efficient File Lifecycle Management**: Providing a seamless interface for the full CRUD (Create, Read, Update, Delete) lifecycle of files, tailored to the authenticated user's context.

- **Optimized Hybrid Storage**: Utilizing a dual-storage strategy where metadata is persisted in a **MySQL** relational database and binary contents are stored on the server filesystem for maximum performance.

- **Reliable Frontend-Backend Integration**: Using **Axios interceptors** to automate security header injection and handle session expiration (401/403 errors) gracefully to maintain a professional user experience.

- **Deployment Readiness**: Creating a scalable architecture that follows the three-layer model (Presentation, Application, and Data) to ensure the system can be easily integrated into larger enterprise CRM environments.

CHAPTER - 2
LITERATURE SURVEY

The literature survey for the **Secure File Management System** in a CRM context examines the integration of modern full-stack development, stateless security protocols, and efficient data persistence strategies.

1. Modern Full-Stack Development in CRM

The evolution of Customer Relationship Management (CRM) tools has shifted toward decoupled architectures that separate the user interface from the business logic. By utilizing **React 18** and **TypeScript 5.x**, developers can build type-safe, responsive interfaces that handle complex customer interactions efficiently. On the server side, **Spring Boot 3.2** serves as a robust application framework, providing the necessary infrastructure for RESTful controllers and service orchestration required for enterprise-level file handling.

2. Stateless Security and JWT Authentication

Security is a foundational requirement for CRM systems handling sensitive customer files. Modern implementations have moved away from traditional session-based storage toward **Stateless JWT (JSON Web Token)** authentication. Using **JJWT**, the system generates tokens containing role claims and expiration data, which are then validated by a security filter chain. This approach ensures that the backend remains scalable while maintaining strict verification of every request via a **Bearer token** model.

3. Role-Based Access Control (RBAC)

To manage user permissions within a CRM, **Role-Based Access Control (RBAC)** is essential to isolate data between different user tiers. In this system, roles are categorized into **ADMIN** and **CUSTOMER**. While customers are restricted to managing their own file context, administrators are granted global visibility into users and system-wide statistics. This dual-layer enforcement occurs at the frontend through protected routes and at the backend through **Spring Security** request matchers.

4. Client-Server Communication and Interceptors

The communication pipeline between the CRM frontend and the backend API is optimized using **Axios**. Centralized **interceptors** are critical for injecting authorization headers into outgoing requests and handling global error responses. For instance, a **401 (Unauthorized)** or **403 (Forbidden)** response automatically triggers a session reset and redirects the user to the login page, ensuring that the CRM .

CHAPTER - 3

SYSTEM ANALYSIS

The system analysis phase evaluates the transition from manual or centralized file handling to the specialized, secure framework required for **File Uploading in Customer Relationship Management**.

3.1 EXISTING SYSTEM & DISADVANTAGES:

Traditional or "existing" file management within legacy CRM environments often relies on basic server-side rendering or non-specialized storage methods that lack granular control.

Existing System Disadvantages:

- **Stateful Session Vulnerabilities**: Older systems often use server-side sessions which are prone to CSRF attacks and do not scale well in modern cloud environments.
- **Lack of Role Isolation**: Many legacy systems do not strictly enforce data boundaries, potentially allowing one user to access another's sensitive documents if the URL is known.
- **Database Bloating**: Storing large binary files directly inside relational database tables (BLOBs) significantly degrades performance and complicates backups.
- **Manual Security Handling**: Without automated interceptors, developers must manually attach authentication credentials to every request, increasing the risk of human error and security leaks.

Existing System Advantages:

While legacy or manual file systems have significant drawbacks, they are often used due to certain perceived benefits:

- **Simplicity of Setup**: Basic systems often require minimal initial configuration compared to complex full-stack architectures.
- **Low Initial Resource Overhead**: Traditional server-side rendering avoids the heavy JavaScript bundling required for modern SPAs.
- **No Client-Side State Management**: Because logic is handled on the server, there is no need to manage complex frontend states or local storage persistence.

3.2 PROPOSED SYSTEM & DISADVANTAGES:

The proposed **Secure File Management System** addresses these flaws by implementing a modern, decoupled architecture using **Spring Boot 3.2** and **React 18**.

Proposed System Disadvantages

- **Increased Architectural Complexity**: Moving from a monolithic, server-side rendered system to a decoupled **React-Spring Boot** architecture increases the complexity of the codebase. Managing two separate repositories, different build tools (Vite vs. Maven), and distinct deployment environments requires higher technical expertise.
- **JWT Management and Security Risks**: While stateless authentication is scalable, it introduces the challenge of **token invalidation**. Since JWTs are stored on the client side (localStorage), they cannot be easily revoked by the server before they expire unless a complex "blacklist" or "refresh token" mechanism is implemented.
- **Client-Side Security Responsibility**: Storing authentication tokens in localStorage makes the application potentially vulnerable to **Cross-Site Scripting (XSS)** attacks if the frontend dependencies are not strictly audited. The security of the system shifts significantly toward the integrity of the client-side code.

- **Initial Development Overhead**: Setting up the security filter chains, CORS configurations, and Axios interceptors requires more upfront development time than a traditional system where the browser handles cookies and sessions automatically.

- **Resource Consumption on Client**: As a Single Page Application (SPA), the system shifts the rendering burden to the user's browser. Users with older hardware or very limited memory may experience slower performance compared to lightweight, server-rendered HTML pages.

- **Dependency on Network Reliability**: Because every action (searching, uploading, deleting) requires an asynchronous API call, the user experience is heavily dependent on low-latency network connections. Unlike traditional systems that might cache full pages, the SPA requires a constant, stable connection to the REST API to remain functional.

Proposed System Advantages

The proposed system utilizes a Spring Boot 3.2 and React 18 stack to provide enterprise-grade advantages for CRM document handling:

- **Stateless Scalability:** By using JWT, the backend does not need to store session data, allowing it to scale horizontally across multiple servers without losing user context.

- **Strict Role-Based Access Control (RBAC):** The system ensures high security by isolating ADMIN and CUSTOMER roles, preventing unauthorized access to sensitive customer files.

- **Automated Security Integration:** Utilizing Axios interceptors ensures that security tokens are automatically attached to every request, reducing the risk of human error in API communication.

- **Optimized Data Performance:** The system separates structured metadata (stored in MySQL) from physical file binaries (stored in the filesystem), ensuring that database performance remains high even as file volume grows.

- **Immediate Feedback & Dynamic UI:** Using React allows for a responsive dashboard where file listings refresh instantly upon upload without requiring a full page reload.

- **Robust Error Handling:** Centralized security controls automatically clear sessions and force re-login during 401/403 errors, protecting the CRM environment from expired or compromised credentials.

CHAPTER - 4
SYSTEM STUDY

The system study involves a detailed evaluation of the "File Uploading in CRM" project to determine if the proposed solution is practical and sustainable for long-term use within an organization.

4.1. ECONOMIC FEASIBILITY:

Economic feasibility determines whether the project is financially viable by comparing the development and operational costs against the potential benefits and savings it provides to the organization. For the **File Uploading in CRM** system, the economic analysis is highly favourable due to the use of open-source technologies.

- **Low Development Costs**: The project is built using open-source frameworks and tools, including **Spring Boot**, **React**, and **MySQL**. This eliminates expensive licensing fees associated with proprietary software suites. Furthermore, the use of **Maven** and **Vite** as build tools ensures that the development environment requires no additional financial investment.
- **Infrastructure & Storage Efficiency**: By implementing a hybrid storage model—where metadata is stored in **MySQL** and binary files are stored on the **server filesystem**—the system avoids the high costs associated with premium cloud database storage for large files. Physical disk storage is significantly more cost-effective than database-managed storage (BLOBs) as the system scales.
- **Reduced Operational Expenses**: The automated nature of the system, particularly the **Role-Based Access Control (RBAC)** and centralized file management, reduces the need for manual administrative labor. Automated session handling via **JWT** reduces the server-side memory overhead, allowing the system to run on standard, cost-efficient hardware or low-tier cloud instances.
- **Security Risk Mitigation**: From an economic standpoint, security breaches are a major financial liability. By utilizing **BCrypt** for password hashing and **JWT** for secure, stateless communication, the system minimizes the financial risks associated with data leaks and unauthorized access to sensitive customer documents.
- **Long-Term Value**: The use of a modular, three-layer architecture ensures that the system can be updated or scaled without requiring a total overhaul, protecting the initial investment

4.2 TECHNICAL FEASIBILITY:

The technical feasibility study evaluates whether the organization has the necessary technological resources and expertise to develop, implement, and maintain the **File Uploading in CRM** system.

- **Robust Frameworks and Language**: The system is developed using **Java 17** and **Spring Boot 3.2** for the backend, which are industry standards for building high-performance, enterprise-grade applications. The frontend utilizes **React 18** and **TypeScript**, providing a type-safe and component-based architecture that is both scalable and easy to debug.
- **Scalable Security Architecture**: The implementation of **Stateless JWT (JSON Web Tokens)** via **Spring Security 6.x** is a technically sound approach. It allows the server to verify user identity and roles without storing session data in memory, making the system capable of handling a growing number of CRM users efficiently.
- **Effective Data Management**: The project adopts a proven hybrid storage strategy. By using **MySQL 8.0** for relational metadata and the **Server Filesystem** for binary files, the system avoids the technical pitfalls of database bloating. This ensures that search queries remain fast and file retrieval remains consistent.
- **Development and Build Tools**: The use of **Maven** for the backend and **Vite** for the frontend ensures a standardized build process. These tools automate dependency management and optimize the application for production, reducing the technical overhead during the deployment phase.
- **Integration Capabilities**: Using **Axios interceptors** on the client side allows for seamless communication with the REST API. This technical choice ensures that security headers are handled centrally, making the integration between the user interface and the secure backend both reliable and maintainable.
- **Infrastructure Requirements**: The system is designed to run on standard hardware and is compatible with modern cloud environments. Its micro-service-ready architecture means that the frontend and backend can be hosted on separate servers (e.g., port 3000 and 8080) to optimize resource distribution.

4.3 SOCIAL FEASIBILITY:

Social feasibility evaluates how the "File Uploading in CRM" system will be accepted by its users and its impact on the social and organizational environment of a business.

- **Data Privacy and User Trust**: In any CRM environment, customers are highly sensitive about the privacy of their documents (such as IDs, contracts, or personal records). The proposed system builds social trust by implementing strict **Role-Based Access Control (RBAC)**. Users feel secure knowing that their files are isolated and inaccessible to other customers, which is essential for user adoption and long-term loyalty.
- **Professional User Experience (UX)**: The interface, developed with **React 18** and styled with **Tailwind CSS**, provides a modern and intuitive experience. By offering a clean dashboard for file management, the system reduces the technical anxiety often associated with complex enterprise software. This makes the transition from manual file handling to digital management smoother for non-technical users.
- **Operational Accountability**: The inclusion of an **ADMIN** role creates a sense of organizational order. Administrators can monitor the volume of data and oversee user activity, ensuring that the platform is used appropriately. This transparency promotes a responsible work culture within the organization.
- **Empowerment through Self-Service**: The system empowers customers by providing them with self-service capabilities—such as independently uploading and retrieving their own documents at any time. This reduces the need for back-and-forth communication with support staff, leading to higher user satisfaction and efficiency.
- **Accessibility and Responsiveness**: Since the frontend is built as a responsive Single Page Application (SPA), users can access their CRM documents from various devices, including tablets and smartphones. This flexibility caters to the modern "work-from-anywhere" social trend, making the system highly relevant to today's workforce.
- **Security Awareness**: By utilizing visible security features like password strength requirements during registration and clear session timeouts (via **JWT** logic), the system educates users on the importance of digital security, fostering a more security-conscious user base.

4.4 OPERATIONAL FEASIBILITY :

Operational feasibility focuses on how effectively the **File Uploading in CRM** system will function within the daily business routine and whether it solves the problems identified in the system analysis.

- **User Role Management**: The system is designed to seamlessly integrate into organizational workflows by defining clear operational boundaries. **ADMIN** users can oversee the entire file repository and monitor system statistics, while **CUSTOMERS** are restricted to managing their own document lifecycle. This prevents operational overlap and ensures data integrity.
- **Automated Session Handling**: From an operational standpoint, the use of **Axios Interceptors** is a major advantage. It automates the attachment of security tokens to requests and handles session expiration (401/403 errors) by automatically redirecting users to the login page. This reduces the burden on the user to manage their own session state and ensures the system remains secure without constant manual monitoring.
- **Simplified File Lifecycle**: The interface provides a streamlined process for uploading, listing, and downloading files. By replacing manual email-based or physical document transfers with a centralized digital portal, the system significantly reduces the "time-to-action" for CRM tasks, making daily operations more efficient.
- **Administrative Oversight**: The system provides a dedicated dashboard for administrators to track total users, total files, and storage metrics. This allows management to make data-driven decisions regarding storage capacity and user engagement, ensuring that the system scales operationally as the business grows.
- **Maintainability and Support**: Built with a **three-layer architecture**, the system is operationally "clean." If a bug occurs in the file-saving logic, it can be fixed in the Service Layer without affecting the User Interface. This separation of concerns minimizes downtime and simplifies the support process for the IT department.
- **Security Compliance**: By enforcing **BCrypt** hashing and **JWT** validation, the system meets standard operational security compliance requirements. This ensures that the organization remains protected against common vulnerabilities while handling sensitive customer documentation.

CHAPTER – 5

TECHNOLOGY STACK

Backend:

Technology | Version | Purpose

Java | 17 | Programming language

Spring Boot | 3.2.x | Application framework

Spring Security | 6.x | Authentication and authorization

Spring Data JPA | 3.x | ORM and repository abstraction

JJWT | 0.12.x | JWT generation and validation

MySQL | 8.x | Relational data storage

Maven | 3.x | Build and dependency management

Frontend:

Technology | Version | Purpose

React | 18.x | UI library

TypeScript | 5.x | Type-safe frontend code

Vite | 5.x | Dev server and bundler

React Router | 6.x | Client-side routing

Axios | 1.x | HTTP client with interceptors

Tailwind + current | Component styling and UI primitives

Runtime Ports:

- Frontend: http://localhost:3000

- Backend API: http://localhost:8080/api

CHAPTER - 6

SYSTEM SPECIFICATIONS

The system specifications outline the minimum and recommended hardware and software requirements necessary to develop, deploy, and run the File Uploading in CRM application effectively.

6.1 HARDWARE REQUIREMENTS:

The following hardware configuration ensures that both the development environment and the production server can handle file processing and concurrent user requests without latency.

- **Processor**: Intel Core i5 or higher (Recommended: i7 or Ryzen 7 for faster compilation).
- **RAM**: Minimum 8 GB (Recommended: 16 GB to run the IDE, Docker, and multiple browser tabs simultaneously).
- **Hard Disk**: 256 GB SSD or higher (SSD is preferred for faster I/O operations during file uploads).
- **Network**: High-speed internet connection for downloading dependencies and API testing.
- **Monitor**: Minimum 1024 x 768 resolution for the React dashboard interface.

6.2 SOFTWARE REQUIREMENTS:

The software stack is built on modern, open-source technologies that provide a secure and scalable environment.

Backend Environment

- **Operating System**: Windows 10/11, macOS, or Linux (Ubuntu 22.04+).
- **Language**: Java 17 (LTS).
- **Framework**: Spring Boot 3.2.
- **Build Tool**: Maven 3.9+.
- **Database**: MySQL 8.0.
- **Security**: Spring Security 6.x with JJWT for token handling.

Frontend Environment

- **Library**: React 18.
- **Language**: TypeScript 5.x.
- **Build Tool**: Vite.
- **Styling**: Tailwind CSS with shadcn/ui components.
- **HTTP Client**: Axios for API integration.

Development Tools

- **IDE**: IntelliJ IDEA (for Backend) and Visual Studio Code (for Frontend).
- **Database Client**: MySQL Workbench or DBeaver.
- **API Testing**: Postman or Insomnia.
- **Version Control**: Git and GitHub.

6.3 ARCHITECTURE & CONNECTIVITY:

The system operates on a **Client-Server Architecture**. The frontend and backend communicate over the HTTP protocol using RESTful endpoints.

- **Frontend Port**: Default is 5173 (Vite) or 3000.
- **Backend Port**: Default is 8080.
- **CORS Policy**: The backend is configured to allow cross-origin requests specifically from the frontend origin to ensure secure data exchange.
- **File Storage**: A dedicated local directory (e.g./uploads) must have read/write permissions for the application user.

CHAPTER - 7

SYSTEM DESIGN

7.1. SYSTEM ARCHITECTURE:

The **File Uploading in CRM** platform is built on a robust, decoupled architecture designed to handle secure data transmission and role-based file management. The system follows a **Three-Tier Architecture** model, ensuring that the presentation, logic, and data management layers remain independent and maintainable.

Key Features:

7.1.1 Presentation Tier (Frontend)

The user interface is a **React 18 Single Page Application (SPA)** that serves as the entry point for all users.

- **Role-Based Routing**: Utilizing React Router, the application implements protected routes that verify the presence of a valid **JWT** before granting access to the Dashboard.
- **State Management**: An AuthContext provider centrally manages user information, roles (**ADMIN** or **CUSTOMER**), and the authentication token retrieved during login.
- **Dynamic UI**: Styled with **Tailwind CSS**, the interface adapts based on the user's role—providing an "Upload" interface for customers and a "System Overview" for administrators.

7.1.2 Application Tier (Backend)

The logic layer is powered by **Spring Boot 3.2**, which acts as the Secure API Gateway.

- **RESTful Controllers**: Expose specific endpoints for user authentication, file uploads, and administrative statistics.
- **Spring Security Filter Chain**: This is the core security mechanism. It intercepts every incoming HTTP request to validate the **Bearer Token** in the Authorization header. If the token is invalid or missing, the request is rejected before it reaches the service layer.
- **Service & Business Logic**: Handles the processing of multipart file uploads, renames files for security, and calculates storage metrics for the admin dashboard.

7.1.3 Data Tier (Persistence)

The storage layer utilizes a hybrid approach to optimize performance and prevent data corruption.

- **Relational Database (MySQL)**: Stores structured data such as user profiles, encrypted passwords (BCrypt), and file metadata (original name, file size, content type, and owner ID).

- **Physical Filesystem**: Binary file contents are saved directly to a secure, non-public directory on the server. This prevents the database from becoming bloated and ensures faster file retrieval through streaming.

7.1.4 Architectural Design Highlights

- **Statelessness**: The server does not store user sessions in memory. This "Stateless" design allows the system to scale easily, as any server instance can validate a JWT independently.

- **Bearer Token Model**: Authentication is handled via a token passed in the request header, which is the industry standard for secure mobile and web communication.

- **Decoupled Communication**: The frontend communicates with the backend solely through JSON and Multipart requests, allowing the two tiers to be updated or replaced independently without breaking the system.

5.Related Diagrams

master-architecture:

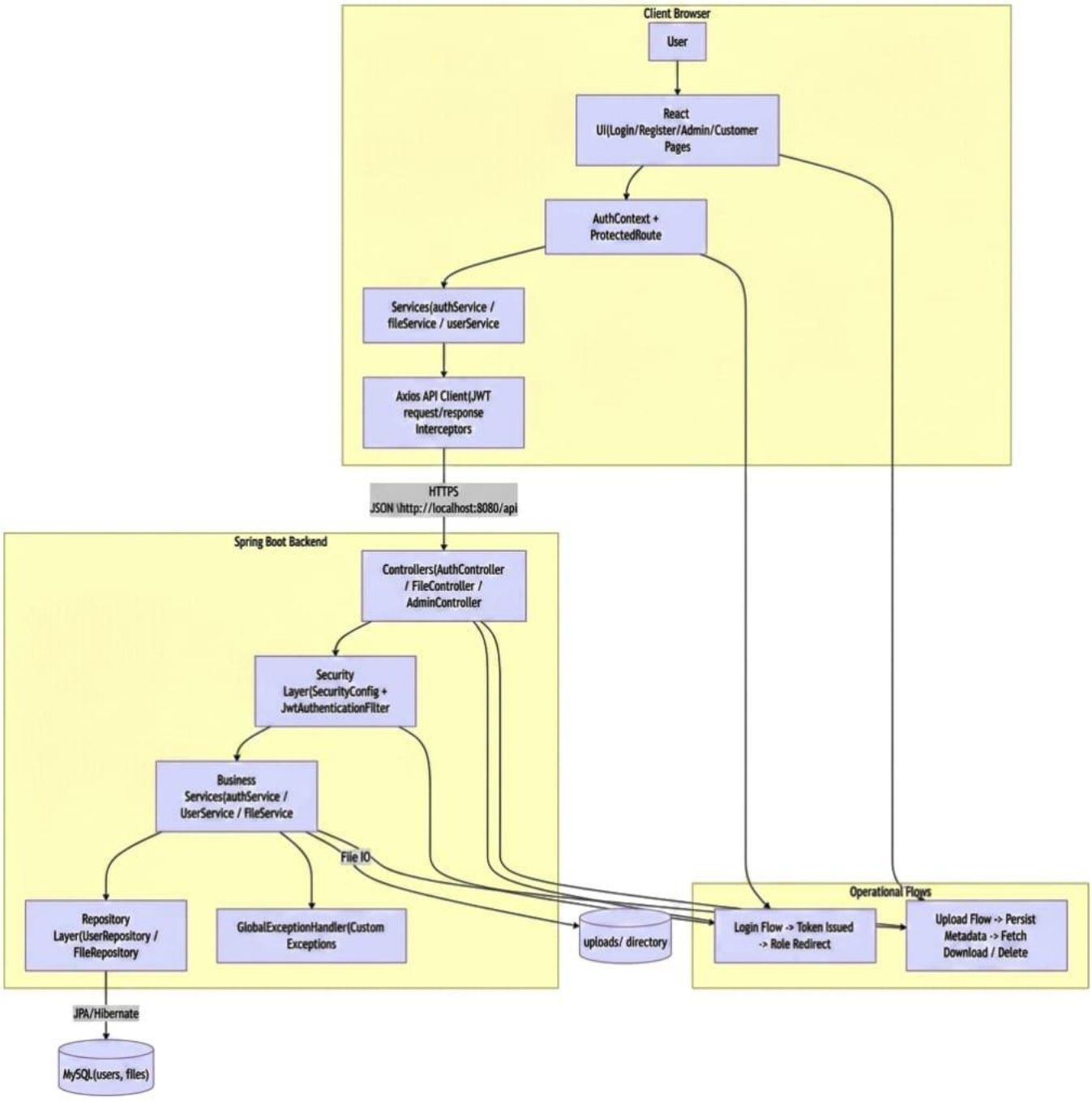


Figure 7.1: Master architecture overview of frontend, backend, security, and storage layers.

Presents the Master Architecture of the CRM system, providing a holistic view of how the four primary layers—Frontend, Backend, Security, and Storage—integrate into a single, cohesive ecosystem. This "bird's-eye view" explains the structural design that supports high-speed file operations and secure user management.

System-architecture-flow:

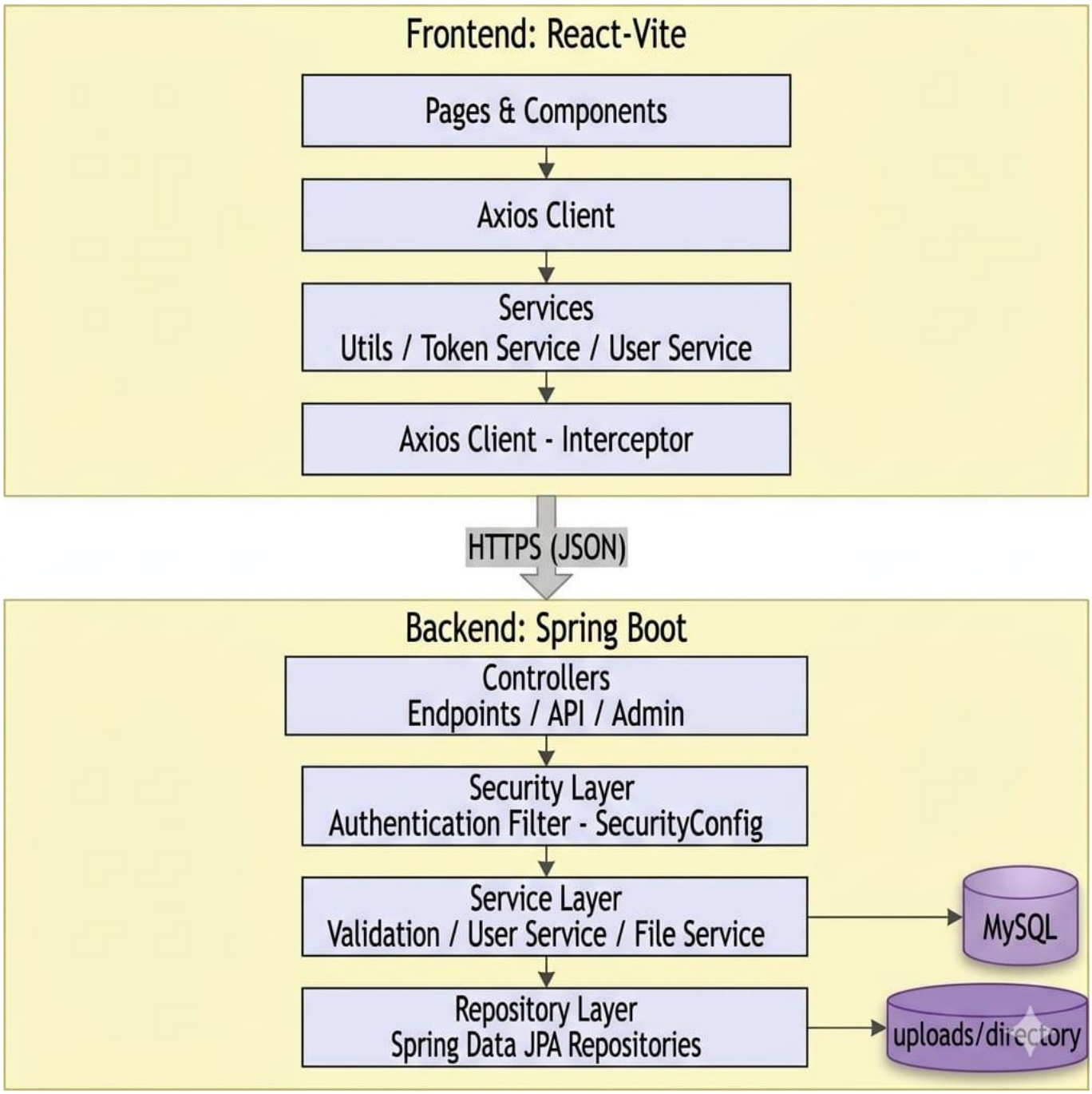


Figure 7.2: End-to-end system architecture flow across application tiers.

Illustrates the End-to-End System Architecture Flow, detailing the sequential movement of data across the three primary application tiers: the Client Tier, the Application Tier, and the Data Tier. This flow ensures that every request—whether a simple login or a complex file upload—is processed with high integrity and security.

Architecture-detailed-view:

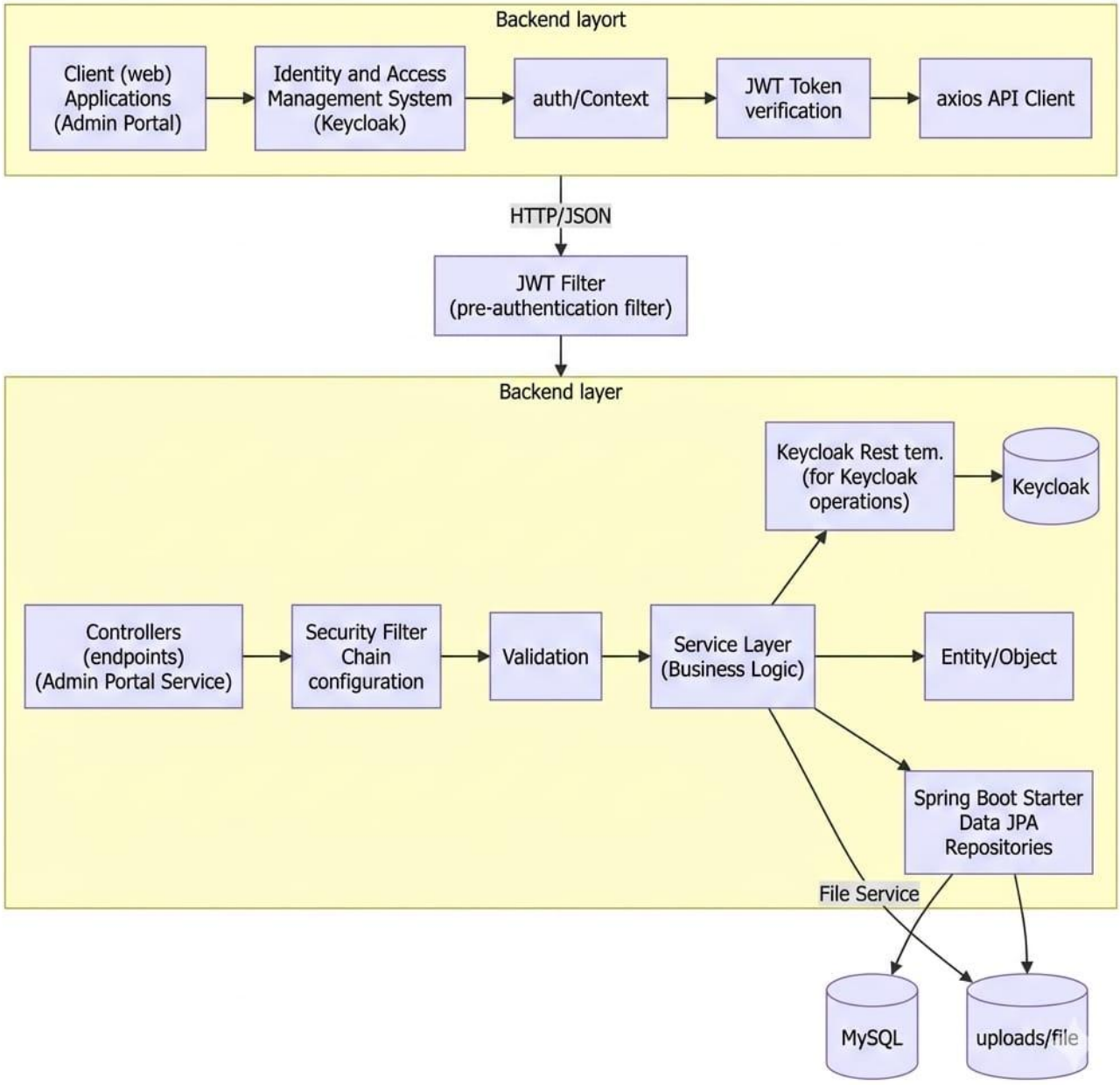


Figure 7.3: Detailed component-level architecture for integration and execution paths.

Illustrates the detailed architecture for the CRM's integration and execution paths. This diagram serves as a technical roadmap, showing how data moves from the user's browser through various security and logic gates before being stored in the database and filesystem.

7.2 DATA FLOW DIAGRAM (DFD):

Data Flow Diagram (DFD) is essential to show how data moves from the user through the security layers and into storage. DFDs are organized by "Levels," moving from a broad overview to specific details.

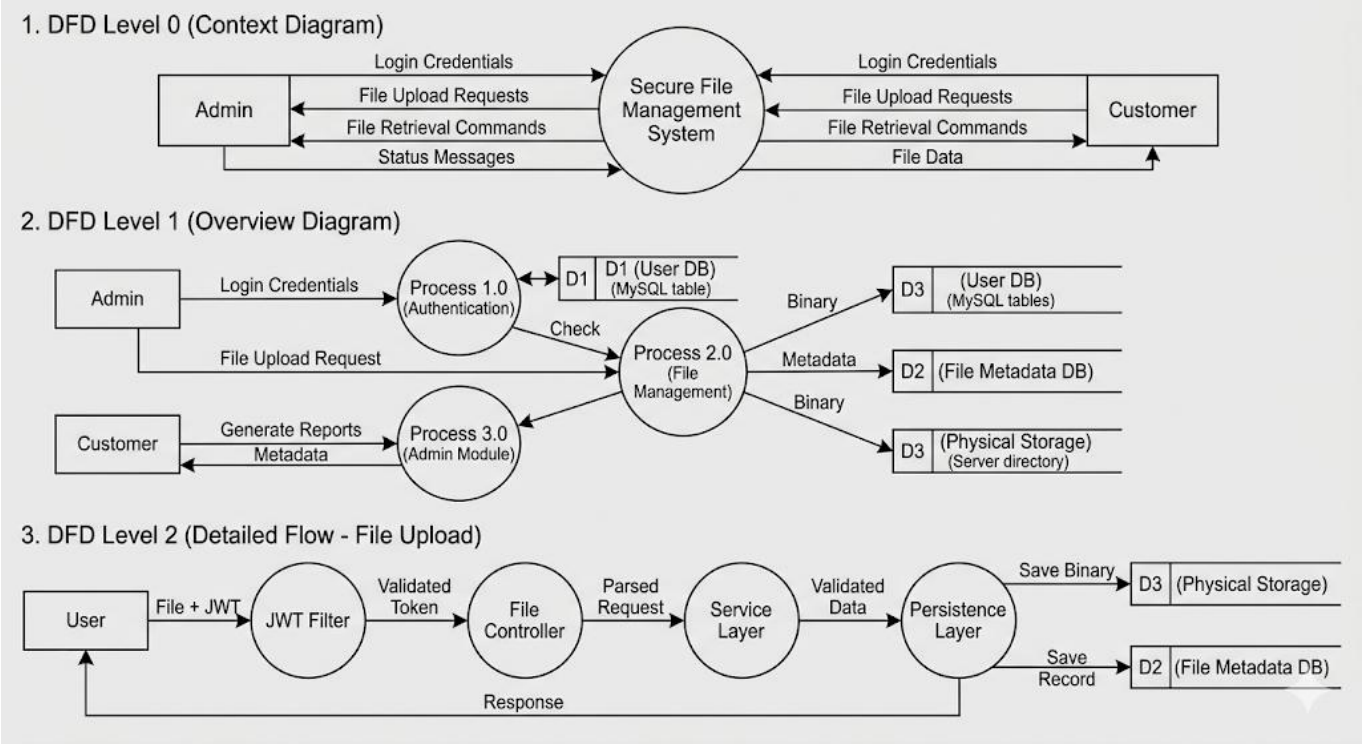


Figure 7.4: Data Flow Diagram (DFD) - Level 0, 1, and 2 for File Uploading in CRM

1. DFD Level 0 (Context Diagram)

This provides a "birds-eye view" of the entire system. It shows the system as a single process and its relationship with external entities (Users).

- **Entities:** Admin, Customer.
- **Process:** Secure File Management System.
- **Data Flows:** Login credentials, file upload requests, and file retrieval commands flow in; status messages and file data flow out.

2. DFD Level 1 (Overview Diagram)

This breaks the system down into its main functional modules. It shows how data is routed between processes and data stores.

- **Process 1.0 (Authentication):** Validates credentials and issues **JWT**.
- **Process 2.0 (File Management):** Handles uploads, downloads, and deletions.
- **Process 3.0 (Admin Module):** Generates reports and manages users.
- **Data Stores:**
    - **D1 (User DB):** MySQL table for user profiles.
    - **D2 (File Metadata DB):** MySQL table for file attributes.
    - **D3 (Physical Storage):** Server directory for binary files.

3. DFD Level 2 (Detailed Flow - File Upload)

This focuses specifically on the logic of a single action, such as **Uploading a File**.

1. **User** sends a file + JWT.
2. **JWT Filter** validates the token.
3. **File Controller** parses the request.
4. **Service Layer** checks for file size/type constraints.
5. **Persistence Layer** saves the binary to the **Filesystem** and the record to **MySQL**.
6. **Response** is sent back to the User.

7.3 USE CASE DIAGRAM:

A **Use Case Diagram** illustrates the dynamic behavior of the system by showing how different "Actors" (users) interact with the "Use Cases" (functional requirements). For the **Secure File Management System**, it defines the boundaries of access between an Administrator and a Customer

- **Customer**: A standard user who manages their personal documents.
- **Admin**: A privileged user who oversees the entire system and all users.

Key Use Cases

- **Authentication Module**: Includes Login and Registration. (Note: Most other use cases <<include>> the Login process for security).
- **File Operations**: Includes Upload File, Download File, View File List, and Delete File.
- **Administrative Operations**: Includes View All Users, Monitor System Statistics, and Global File Oversight.

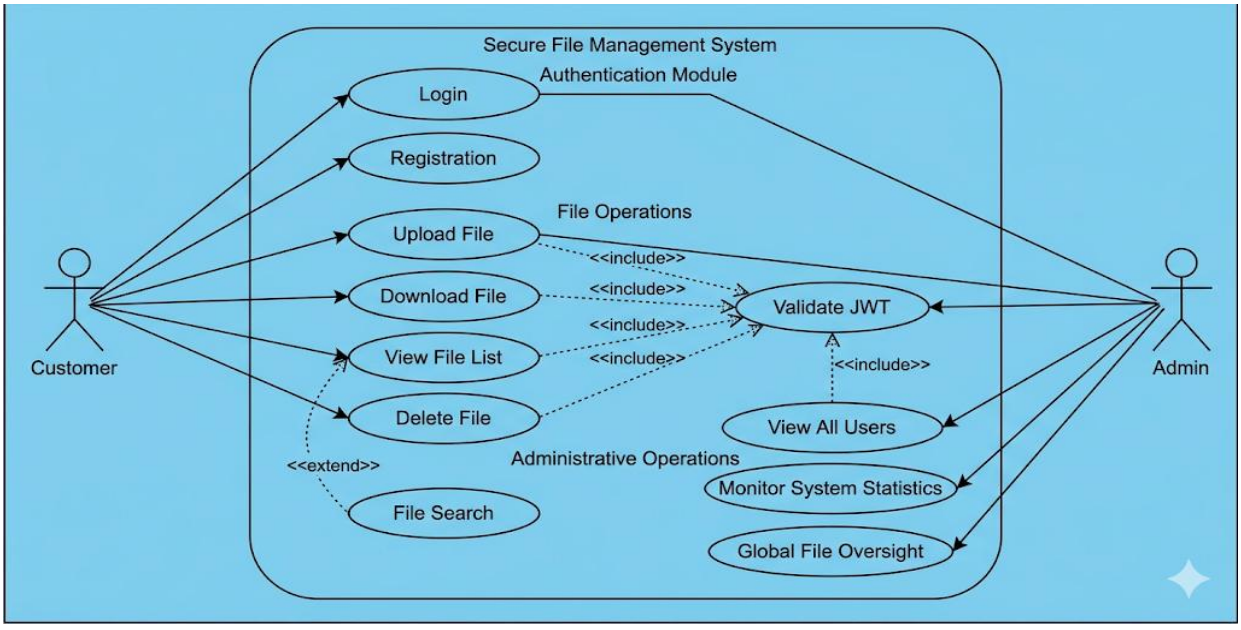


Figure 7.5: Use Case Diagram for File Uploading in CRM

Use Case Relationships

- **Include (<<include>>)**: Actions like uploading or deleting a file "include" the Validate JWT process, meaning the system must verify the token before the action occurs.
- **Extend (<<extend>>)**: An optional action, such as "File Search," which extends the basic "View File List" functionality.

7.4. CLASS DIAGRAM:

The **Class Diagram** is a static structural diagram that describes the structure of the system by showing its classes, attributes, operations (or methods), and the relationships among objects.

In this CRM system, the architecture follows the **Object-Relational Mapping (ORM)** pattern, where Java classes in the backend correspond directly to tables in the **MySQL** database.

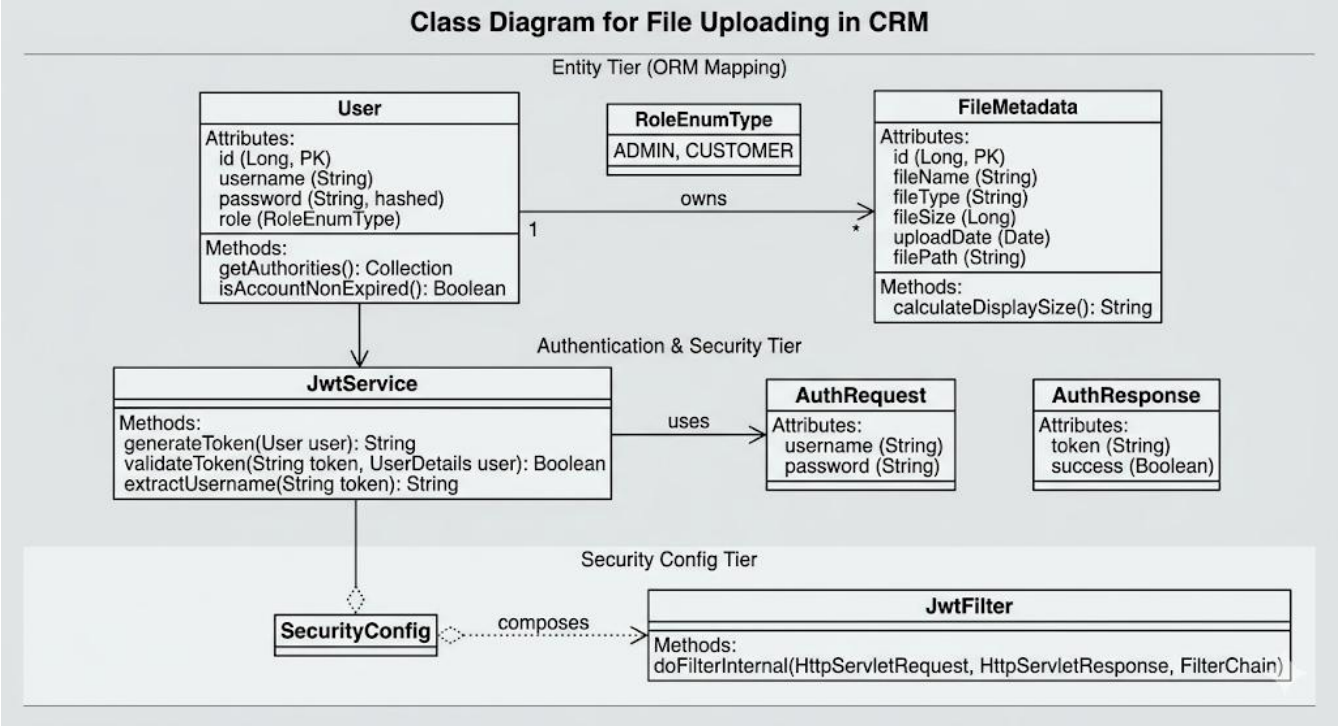


Figure 7.6: Class Diagram for File Uploading in CRM

Core Components

1. User Entity

This class represents the system users. It stores identity and security information.

- **Attributes:** id, username, password (hashed), role (Enum: ADMIN, CUSTOMER).
- **Methods:** getAuthorities(), isAccountNonExpired().

2. File Meta data Entity

This class tracks the details of the files uploaded to the server. Note that it does not store the file itself, but the information *about* the file.

- **Attributes:** id, fileName, fileType, fileSize, uploadDate, filePath.
- **Methods:** calculateDisplaySize().

3. Authentication Components

These classes handle the security logic.

- **Jwt Service:** Methods like generateToken(), validateToken(), and extractUsername().
- **AuthRequest/AuthResponse:** Data Transfer Objects (DTOs) used to move login data between the React frontend and Spring Boot backend.

Relationships

- **One-to-Many ():** A single **User** can be associated with multiple **File Meta data** records. Each file, however, belongs to exactly one user.

- **Composition:** The **Security Config** class "composes" the **JwtFilter**, meaning the filter is a functional part of the overall security configuration.
- **Static Structure:** The diagram organizes the system into specific tiers (Entity, Authentication, and Security Config), depicting the classes as backend components with attributes, methods, and relationships.
- **Object-Relational Mapping (ORM):** The `User` and `FileMetadata` classes are depicted with standard Java data types and database keys (e.g., Long, PK), clearly showing how these Java entities will map to your MySQL database schema.
- **Relationships:** You can see the explicit one-to-many relationship (owns) between a `User` and their files, as well as the composition relationship showing how the `JwtFilter` is a functioning part of the `SecurityConfig`.

7.5 SEQUENCE DIAGRAM:

A **Sequence Diagram** is a behavioral UML diagram that illustrates how objects interact in a specific time sequence. For this project, the most critical flow is the Secure File Upload Process, which demonstrates how the React frontend, Spring Security filters, and the Backend services collaborate.

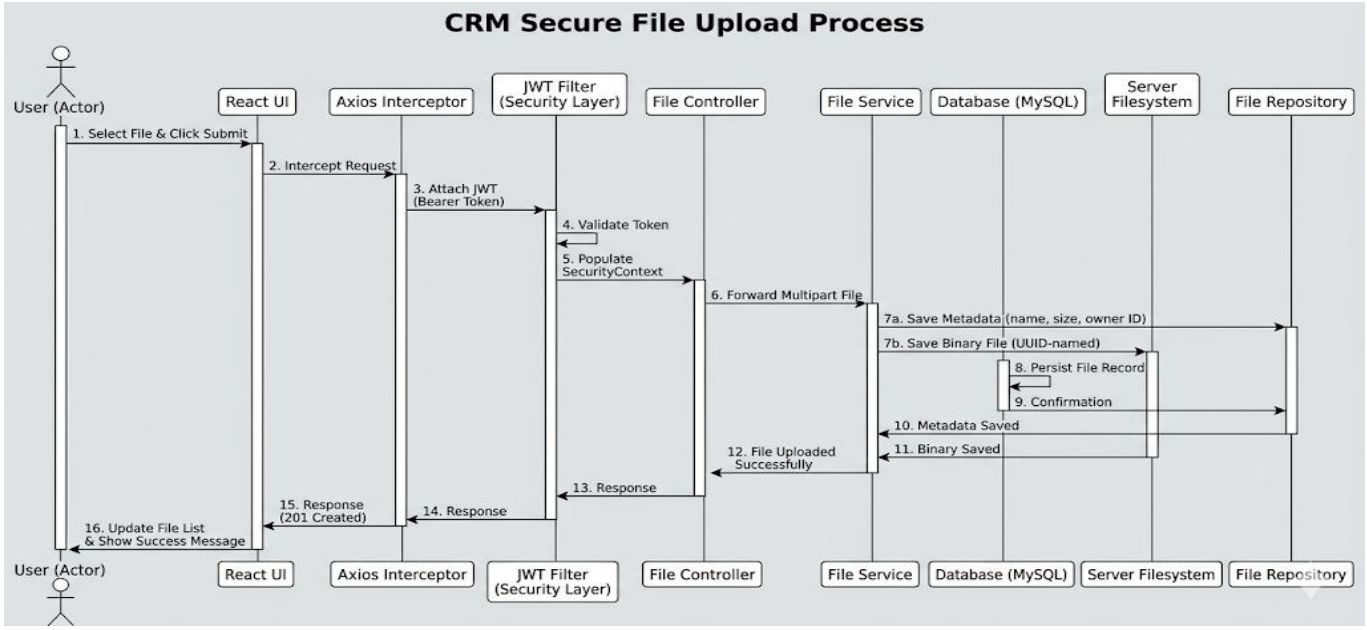


Figure 7.7: Sequence Diagram - CRM Secure File Upload Process

File Upload Sequence Flow

1. **Actor (User)**: Initiates an upload by selecting a file and clicking "Submit" on the React UI.
2. **React Frontend**: Intercepts the request using an **Axios Interceptor** to attach the JWT (Bearer Token) to the header.
3. **JWT Filter (Security Layer)**: Intercepts the incoming request. It validates the token's integrity and expiration. If valid, it populates the SecurityContext.

4. **File Controller**: Receives the Multi part File. It extracts the current user's identity from the security context.
5. **File Service**:
    - Generates a unique filename (UUID) to prevent collisions.
    - Saves the binary file to the **Server Filesystem**.
    - Sends the metadata (name, size, owner ID) to the **File Repository**.
6. **Database (MySQL)**: Persists the file record.
7. **Response**: A success message (201 Created) flows back through the layers to the React UI, which updates the file list for the user.

7.6 ACTIVITY DIAGRAM:

An **Activity Diagram** is essentially a sophisticated flowchart that represents the operational workflow of the system. It focuses on the flow of control and the conditions (decisions) that lead to different outcomes.

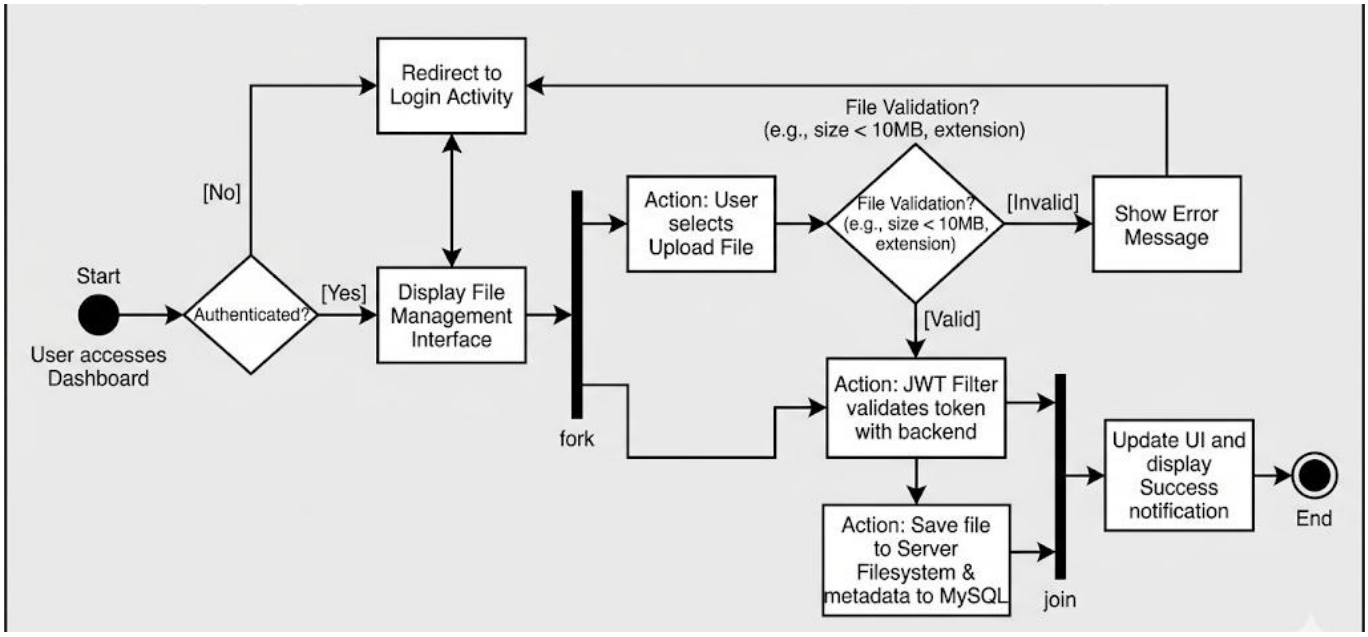


Figure 7.8: Activity Diagram - CRM Secure File Upload Process

Workflow Logic: File Upload & Validation

The following steps describe the typical activity flow when a user attempts to interact with the system:

1. **Start**: User accesses the Dashboard.
2. **Decision (Authenticated?)**: The system checks if a JWT exists in local storage.
    - *If No*: Redirect to **Login Activity**.
    - *If Yes*: Display the **File Management Interface**.
3. **Action**: User selects "Upload File."
4. **Decision (File Validation)**: The system checks if the file meets criteria (e.g., size < 10MB, allowed extension).
    - *If Invalid*: Show **Error Message** and return to dashboard.
    - *If Valid*: Proceed to **Security Interception**.
5. **Action**: The **JWT Filter** validates the token with the backend.
6. **Action**: The system saves the file to the **Server Filesystem** and the metadata to **MySQL**.
7. **End**: Update UI and display "Success" notification.

7.7 HIGH LEVEL ARCHITECTURE:

The High-Level Architecture (HLA) defines the conceptual framework of the **Secure File Management System**. It illustrates how the various distributed components—the Client, the Security Gateway, the Application Logic, and the Data Stores—work together as a unified ecosystem.
The system is built on a **Decoupled Client-Server Model**, allowing the frontend and backend to evolve independently while communicating via a standardized REST API.

The Four Core Layers

1. Client Tier (The Consumer)

- **Technology**: React 18 / TypeScript.
- **Role**: Handles the Presentation Logic and User State.
- **Key Function**: It captures user input (credentials or files) and uses **Axios** to transmit data. It is "Stateless" in terms of the server, meaning it carries its own identity via a stored JWT.

2. Security & Gateway Tier (The Protector)

- **Technology**: Spring Security 6.x + JJWT.
- **Role**: Acts as the "Border Control" for the application.
- **Key Function**: Every request hits the **JWT Filter** first. This layer handles **Authentication** (who are you?) and **Authorization** (what are you allowed to do?). It ensures that a "Customer" cannot access "Admin" endpoints.

3. Application Business Tier (The Processor)

- **Technology**: Spring Boot / Java 17.
- **Role**: Contains the "Brains" of the CRM.
- **Key Function**:
    - **Controllers**: Route requests to the correct logic.
    - **Services**: Handle the heavy lifting, such as file streaming, UUID generation for filenames, and business rules.
    - **Repositories**: Interface with the data stores using Spring Data JPA.

4. Persistence Tier (The Vault)

- **Technology**: MySQL 8.0 + Local/Cloud Filesystem.
- **Role**: Ensures data longevity and integrity.
- **Hybrid Storage Strategy**:
    - **MySQL**: Manages "Light" data (User accounts, file metadata, relationship mappings).

7.8 DATABASE / DOMAIN DESIGN:

The database and domain design for the **File Uploading in CRM** system is structured to ensure data integrity, security, and high performance. The design follows a relational model where file metadata is decoupled from the actual file binaries.

1.Database Schema

Users Table

- id BIGINT PK
- email VARCHAR UNIQUE
- password VARCHAR
- first_name VARCHAR
- last_name VARCHAR
- role VARCHAR
- avatar VARCHAR
- created_at TIMESTAMP
- updated_at TIMESTAMP

2.Files Table

- id BIGINT PK
- name VARCHAR
- type VARCHAR
- size BIGINT
- file_path VARCHAR
- uploaded_at TIMESTAMP
- user_id BIGINT FK -> users.id

3. Entity Relationship (ER) Model

The system architecture revolves around two primary entities: **User** and **FileMetadata**. These entities are linked via a **One-to-Many ()** relationship, meaning one user can own multiple files, but each file record belongs to exactly one user.

4. Data Dictionary (Schema Definition)

The following tables define the structure of the persistence layer in **MySQL**:

Table: Users

Field	Data Type	Constraints	Description
id	BIGINT	PK, Auto-Inc	Unique identifier for the user.
username	VARCHAR(50)	Unique, Not Null	The user's login identifier.
password	VARCHAR(255)	Not Null	**BCrypt** hashed password.
role	ENUM	Not Null	ROLE_ADMIN or ROLE_CUSTOMER.

7.9 CORE FLOW DESIGNS:

The **Core Flow Designs** illustrate the step-by-step logic of the most vital operations within the CRM system. These flows ensure that data moves securely between the user's browser and the server storage while maintaining strict authentication standards.

1. User Authentication Flow

This flow handles the transition from an anonymous visitor to an authenticated user by issuing a **JSON Web Token (JWT)**.

- **Trigger**: User submits the login form with credentials.
- **Process**:
    1. The backend receives the AuthRequest.
    2. The AuthenticationManager verifies the username and password against the **BCrypt** hashes in MySQL.
    3. Upon success, the JwtService generates a signed token containing the user's ID and Role.
- **Outcome**: The token is sent to the React frontend and stored in localStorage, enabling access to protected routes.

2.Secure File Upload Flow

This is the central process of the application, coordinating security, file I/O, and database persistence.

- **Step 1: Interception**: The React **Axios Interceptor** attaches the JWT to the request header.
- **Step 2: Validation**: The Spring Security **JWT Filter** validates the token. If invalid, it returns a 401 error.
- **Step 3: Processing**: The FileService validates the file type/size and generates a unique **UUID filename**.
- **Step 4: Storage**:
    - The binary file is written to the /uploads directory.
    - The metadata (original name, size, owner) is saved to the **MySQL** database.
- **Step 5: Confirmation**: A success response triggers a state refresh in the React UI.

3. File Retrieval & Download Flow

This flow ensures that users can only download files they are authorized to see.

- **Verification**: The system checks the user_id of the requested file against the user_id extracted from the requester's JWT.
- **Streaming**: Instead of loading the entire file into memory, the backend uses a **Resource Stream** to send the file bytes to the client.
- **Cleanup**: The stream is closed once the browser confirms the download is complete.

4. Admin Oversight Flow

A specialized flow for users with the **ROLE_ADMIN** credential.

- **Aggregate Query**: The Admin Dashboard triggers an API call that performs a COUNT and SUM operation on the File_Metadata table.
- **Data Visualization**: The backend returns a JSON object containing global stats (Total Users, Total Files, Storage Used), which the React frontend renders into charts or metric cards

CHAPTER - 8

IMPLEMENTATION / MODULES

The implementation phase of the **File Uploading in CRM** project is organized into modular components. This modularity ensures that the authentication system, file handling logic, and administrative tools can be developed, tested, and maintained independently.

8.1 AUTHENTICATION MODULE:

The **Authentication Module** is the foundational security layer of the CRM. It manages user identity, ensures data privacy through encryption, and provides a stateless session mechanism using **JSON Web Tokens (JWT)**.

Authentication & JWT Flow

Authentication Sequence

1. User submits credentials to /auth/login.
2. Backend validates credentials via authentication manager.
3. JwtUtil generates JWT containing role claim and expiry.
4. Frontend stores token and user payload in localStorage.
5. User is redirected to /admin or /dashboard based on role

Validation Sequence

1. Axios request interceptor attaches Bearer token.
2. JwtAuthenticationFilter validates signature and expiration.
3. SecurityContext is set with authenticated principal.
4. Request reaches controller under role constraints.

Token Handling Notes

- Token is attached automatically from localStorage.
- Invalid/expired token triggers session reset and redirect to /login.

Authentication and JWT Flow

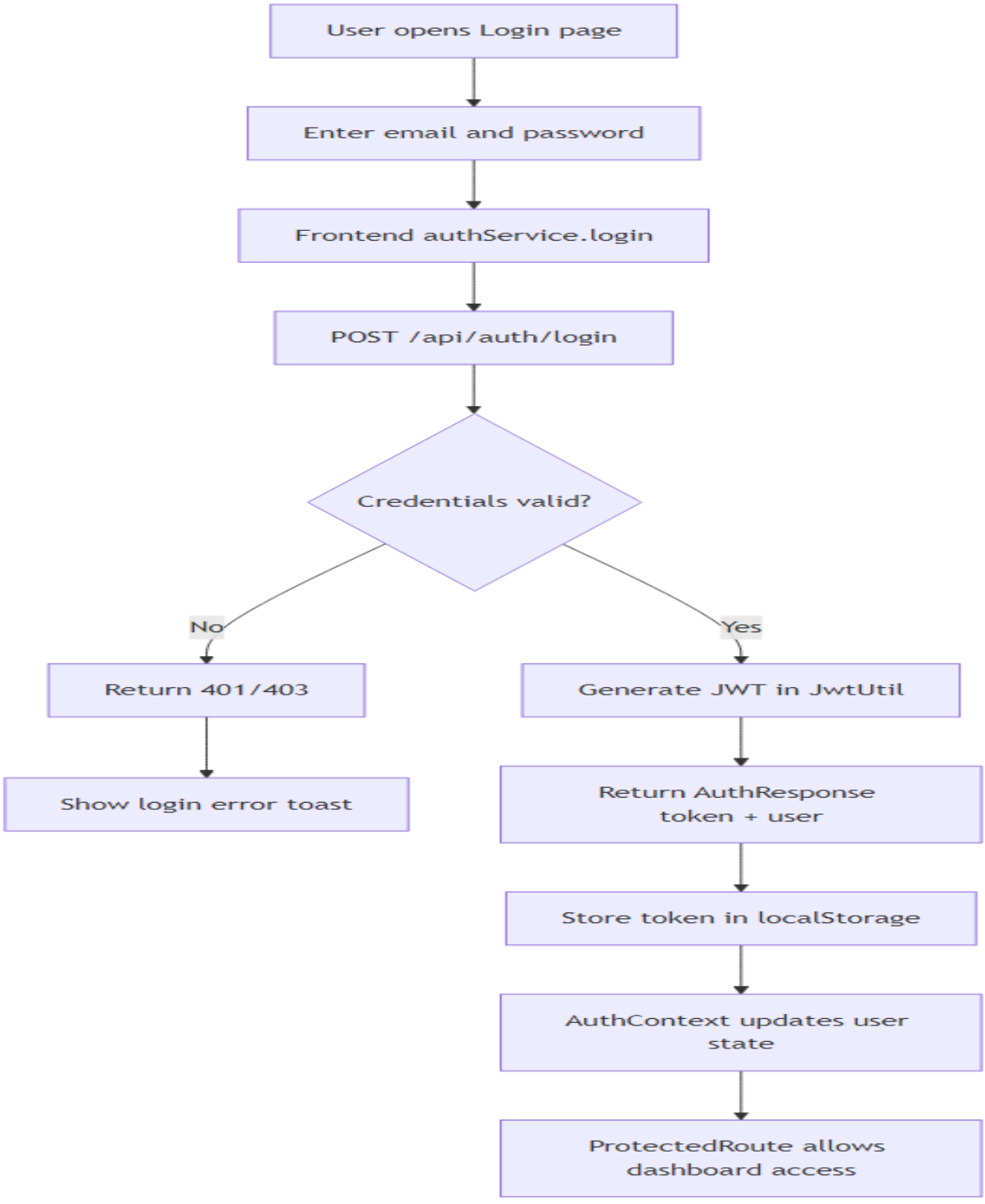


Figure 8.1: Authentication and JWT token lifecycle from login to protected requests

Core Components & Logic

8.1.1. Registration and Password Hashing

When a new customer or admin signs up, the system does not store plain-text passwords.

- **Technology**: **BCrypt Password Encoder**.
- **Logic**: The module applies a "salting" and "hashing" algorithm to the password. Even if the database is compromised, the actual passwords remain unreadable.

8.1.2. The Login Process (Authentication)

This is the "handshake" between the user and the server.

- **Input**: Username and Password.
- **Verification**: The AuthenticationManager compares the input with the hashed value in the **MySQL** database.
- **Token Issuance**: Upon successful verification, the backend generates a **JWT**. This token contains "claims" such as the user's ID and their specific role (ROLE_ADMIN or ROLE_CUSTOMER).

8.1.3. Authorization (Role-Based Access Control)

Once a user is authenticated, the module determines what they are allowed to do.

- **Customer Access**: Restricted to their own file uploads and profile settings.
- **Admin Access**: Granted access to the global dashboard, user management, and system-wide file statistics.
- **Implementation**: Handled via Spring Security's @PreAuthorize annotations on specific API endpoints.

8.1.4. The JWT Security Filter

Every request sent from the React frontend after login must include the token in the Authorization header.

- **Interception**: A custom JwtAuthenticationFilter intercepts every incoming HTTP request.
- **Validation**: It checks if the token is expired, if the signature is valid, and if the user exists.

8.2 USER MODULE:

The **User Module** serves as the administrative and profile management core of the CRM. While the Authentication Module handles the *process* of logging in, the User Module manages the *data* and *lifecycle* of the individuals using the platform.

Core Responsibilities

8.2.1. Profile Management

Every user (Customer or Admin) has a profile that stores their specific identity within the system.

- **Data Fields**: Includes the username, email, full name, and creation date.
- **Relationship Mapping**: This module acts as the "anchor" for all other data. Since files are linked to a UserID, this module ensures that when a user is viewed, all their associated documents can be retrieved via a JPA relationship.

8.2.2. Administrative Oversight (Admin Tier)

For users with ROLE_ADMIN, this module provides elevated capabilities to manage the CRM ecosystem.

- **User Listing**: Fetches a paginated list of all registered accounts.
- **Account Status**: Allows admins to enable, disable, or delete accounts.
- **Activity Monitoring**: Admins can see how many files a specific user has uploaded and how much storage they are occupying.

8.2.3. Role-Based Access Control (RBAC) Logic

The User Module defines the boundaries of what a user is.

- **Customer Role**: Limited to a "Self-Service" scope. They can only see and edit their own data.
- **Admin Role**: Has a "Global" scope. They can see everyone’s data but are usually restricted from seeing private passwords (which are hashed).

8.3. ADMIN MODULE:

The **Admin Module** is the command center of the CRM. It provides high-level visibility and management tools that are strictly inaccessible to standard customers. This module leverages **Administrative Privileges** to oversee system health, manage users, and monitor data storage.

Core Functionalities

8.3.1. System-Wide Dashboard

The primary interface for the Admin is a dashboard that aggregates data from across the entire application.

- **Metric Cards**: Displays real-time statistics such as Total Users Registered, Total Files Uploaded, and Total Storage Occupied.
- **Data Aggregation**: The module executes specialized JPQL (Java Persistence Query Language) queries to sum file sizes and count records without fetching every individual object into memory.

8.3.2. User Management & Control

Admins have the authority to manage the lifecycle of CRM accounts.

- **User Directory**: A paginated list showing all users, their roles, and their registration dates.
- **Account Actions**: Capabilities to deactivate accounts or promote a "Customer" to an "Admin" role if required.
- **User Auditing**: Admins can click on a specific user to view a curated list of all files that particular individual has uploaded.

8.3.3. Global File Oversight

Unlike the Customer module, which is restricted by ownership, the Admin module can access the global file repository.

- **Compliance Monitoring**: Admins can review files to ensure they meet company or legal guidelines.
- **Global Search**: Search for files by name, extension, or uploader across the entire database.
- **Storage Maintenance**: Ability to delete orphaned files or clear out data from deactivated accounts to save server space.

8.4. UPLOAD MODULE:

The **Upload Module** is the functional engine of the CRM, specifically engineered to handle the transfer of binary data from the client to the server. It manages the complexities of multipart requests, file validation, and physical storage pathing.

Core Components

8.4.1. Multipart Request Handler:

Since files are significantly larger and more complex than standard JSON data, the module utilizes the **MultipartFile** interface provided by Spring Boot.

- **Stream Handling**: It processes the incoming byte stream, ensuring the server doesn't hit memory limits when multiple users upload files simultaneously.
- **Temporary Storage**: Files are briefly held in a system temp directory before being moved to their permanent secure location.

8.4.2. Validation Engine

To protect the server from malicious uploads and storage bloat, every file must pass through a strict validation gate.

- **Type Filtering**: Checks the MIME type (e.g., application/pdf, image/png) against an allow-list.
- **Size Constraints**: Rejects files that exceed the pre-defined limit (e.g., 10MB) to prevent "Denial of Service" (DoS) attacks via storage exhaustion.
- **Filename Sanitization**: Cleans the original filename to prevent "Directory Traversal" attacks, where a user might try to save a file into a sensitive system folder using paths like ../../etc/passwd.

8.4.3. Storage Service (I/O)

This component handles the physical interaction with the server's hard drive.

- **UUID Masking**: Every file is renamed to a **Universally Unique Identifier** (e.g., a1b2-c3d4...pdf). This ensures that if two customers upload "Invoice.pdf", they do not overwrite each other.
- **Path Resolution**: Generates the absolute path where the file will reside, typically in a directory outside the application’s public web root for maximum security.

8.4.4.Technical Workflow

1. **Request**: The React frontend sends a POST request with Content-Type: multipart/form-data.
2. **Security Check**: The module identifies the uploader via the JWT to link the file to the correct user_id.
3. **IO Operation**: The FileService writes the bytes to the target directory.

8.5 FILE CRUD OPERATIONS:

Create (Upload)
- Endpoint: POST /files/upload
- Input: multipart/form-data with file
- Output: FileDto

Read (List/User)
- Endpoint: GET /files/user
- Output: FileDto[] scoped to current authenticated user

Read (Single)
- Endpoint: GET /files/{id}
- Output: FileDto

Delete
- Endpoint: DELETE /files/{id}
- Output: 200 OK or error payload

Download
- Endpoint: GET /files/download/{id}
- Output: binary stream with content-disposition attachment header

8.5.1 File Upload & Access Flow:

1. User selects file from dashboard files page.
2. Frontend builds FormData and calls upload endpoint.
3. Backend validates token and resolves user context.
4. File is stored to server path, metadata persisted in MySQL.
5. Response returns FileDto and UI refreshes listing

File Upload Flow

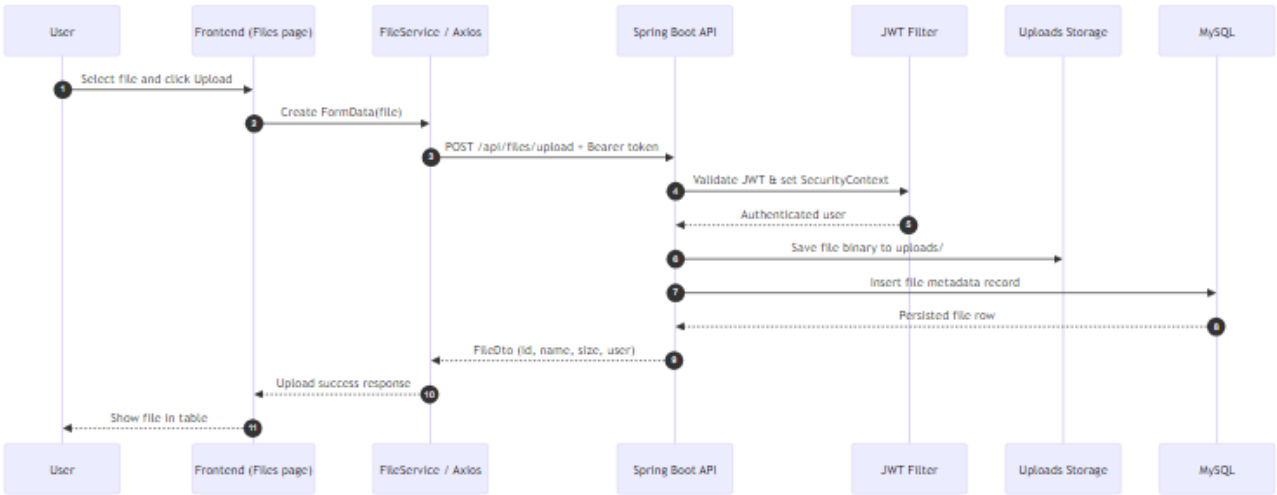


Figure 8.2: File upload, persistence, and response flow for authenticated users

CHAPTER - 9

SOFTWARE ENVIRONMENT

The software environment defines the collection of tools, languages, and platforms used to develop, build, and run the **File Uploading in CRM** system. It is divided into the development environment (tools used by the programmer) and the runtime environment (the stack required to execute the application).

9.1. DEVELOPMENT & BUILD TOOLS:

The development and build environment consists of the specialized software and utilities required to write, compile, and package the CRM application. These tools ensure that the transition from source code to a functional, production-ready system is seamless and error-free.

9.1.1. Integrated Development Environments (IDEs)

IDEs provide a centralized workspace for coding, debugging, and project management.

- **IntelliJ IDEA:** The primary IDE used for the **Spring Boot backend**. It offers robust support for Java, automated refactoring, and deep integration with Maven and Spring Security.
- **Visual Studio Code (VS Code):** Used for the **React frontend** development. It is preferred for its lightweight nature and extensive plugin ecosystem for JavaScript, TypeScript, and CSS.

9.1.2. Build & Dependency Management

These tools automate the process of fetching external libraries and compiling the project into its final format.

- **Apache Maven:** The "backbone" of the backend. Maven uses a pom.xml file to manage dependencies such as **Spring Data JPA**, **JJWT**, and **BCrypt**. It handles the compilation of Java code into an executable .jar file.
- **NPM (Node Package Manager):** The equivalent for the frontend. It manages all React-related packages (e.g., Axios, React Router) and coordinates the build process that transpiles modern JavaScript into browser-compatible code.

9.1.3. API Development & Testing Tools

Before the frontend and backend are linked, these tools are used to verify that the "Upload" and "Auth" logic works as intended.

- **Postman:** A crucial tool used to simulate client requests. It allows developers to test the **Multipart Upload** endpoints, verify that JWT tokens are correctly issued, and ensure that restricted routes return a 403 Forbidden status for unauthorized users.

9.2 FRONTEND ENVIRONMENT:

The frontend environment is designed to provide a highly interactive, responsive, and secure user interface for the CRM. It focuses on the **Single Page Application (SPA)** paradigm, where the page never reloads, providing a smooth "app-like" experience for users managing their files.
The client-side environment focuses on delivering a responsive and interactive user experience.

- **Core Library: React 18**. A component-based library used to build the dynamic user interface.
- **Communication: Axios**. An HTTP client used to send multi-part data (files) and handle the injection of JWT tokens into request headers.
- **Styling: Tailwind CSS / Bootstrap**. Used to create a clean, professional CRM dashboard that adapts to different screen sizes.

9.2.1 Frontend Pages & Components

Primary Routes

- /login
- /register
- /admin
- /admin/users
- /admin/files
- /dashboard
- /dashboard/files
- /dashboard/profile

Key Components

- App Layout
- App Sidebar
- Protected Route
- Auth Context provider

User Interface Diagrams:

User Interface Login

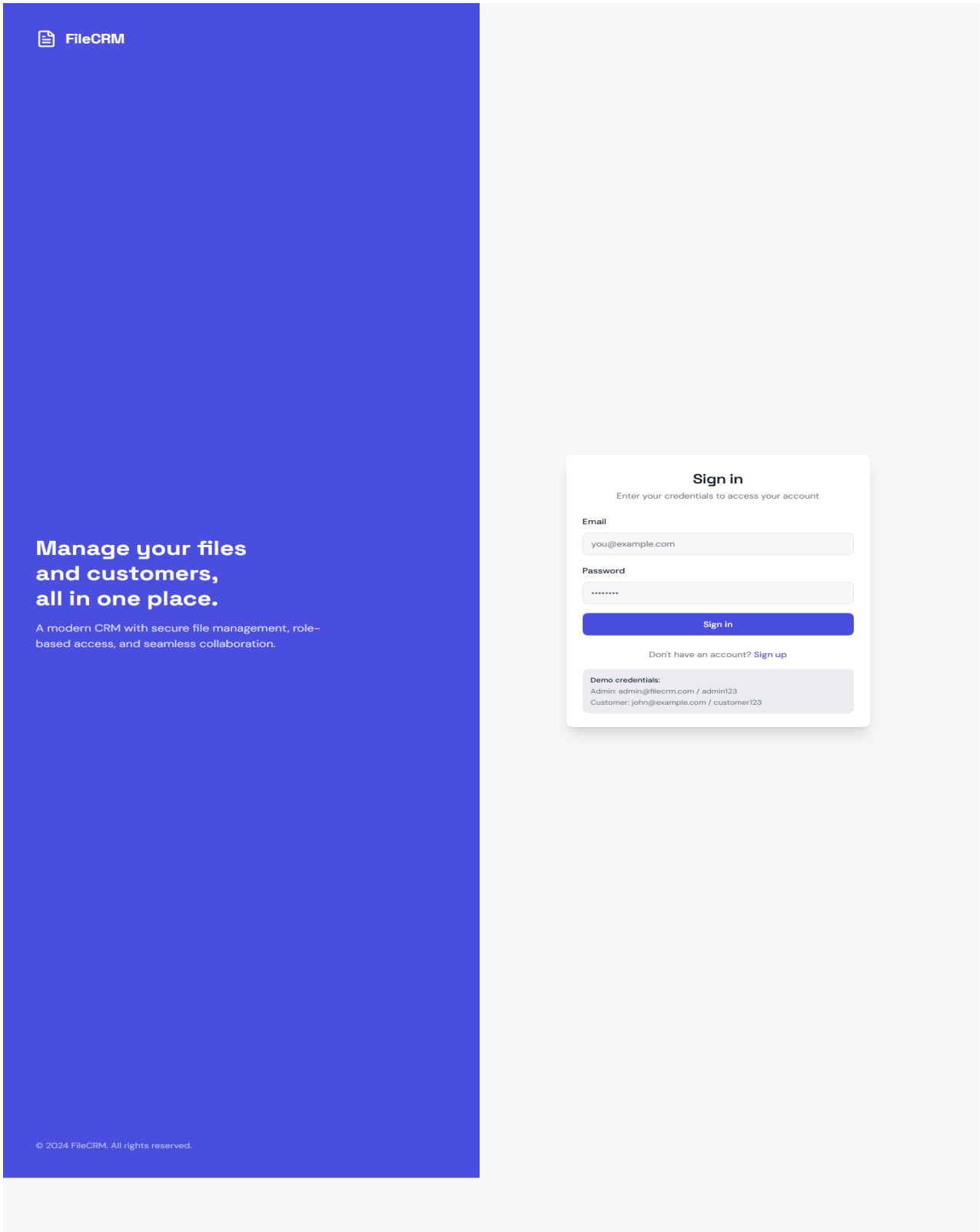


Figure 9.1: Login screen used for credential-based access.

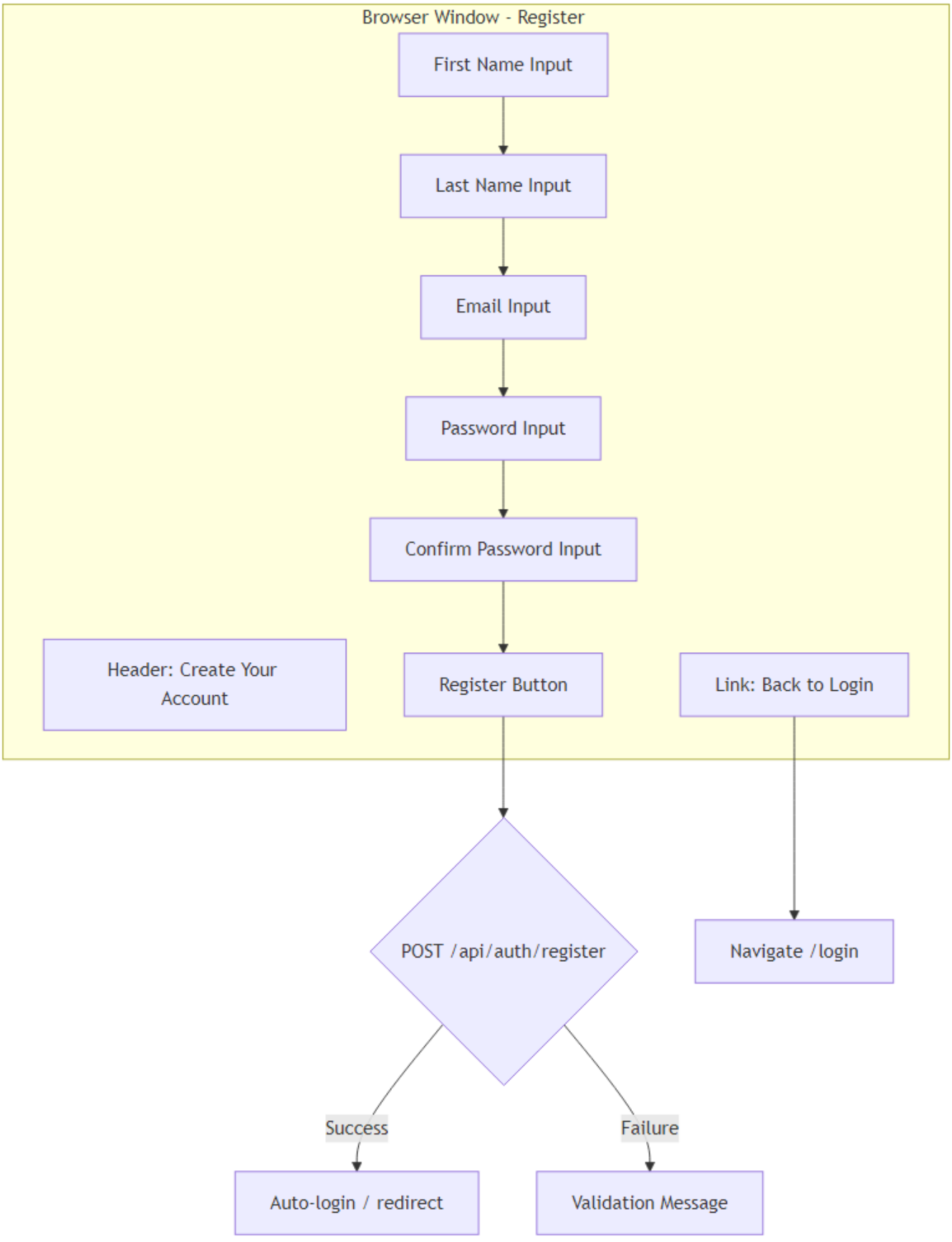


Figure 9.2: Registration screen for onboarding new users.

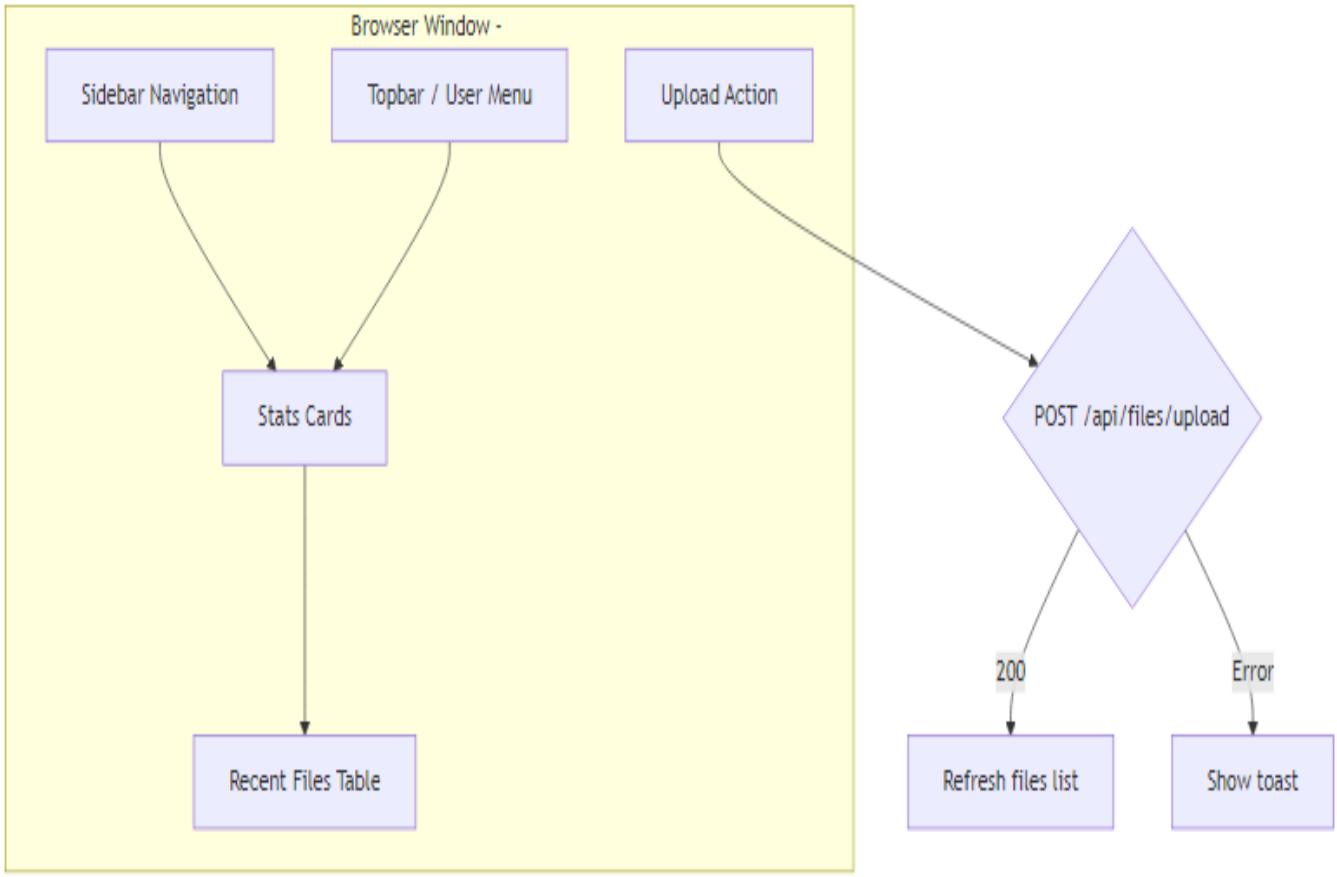


Figure 9.3: Root application view after route resolution.

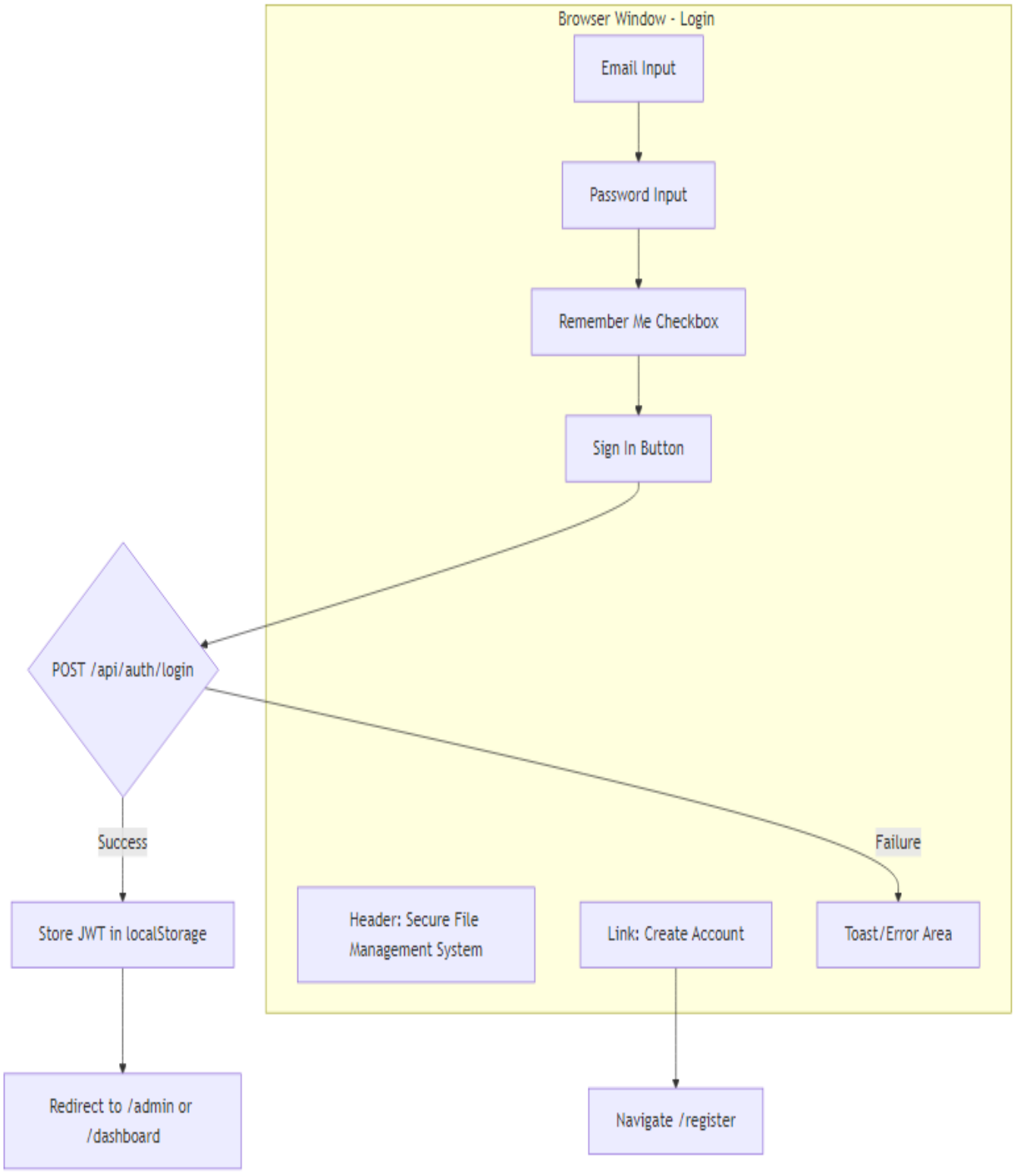


Figure 9.4: Structural UI diagram of login page components and flow.

User Interface Register Screen

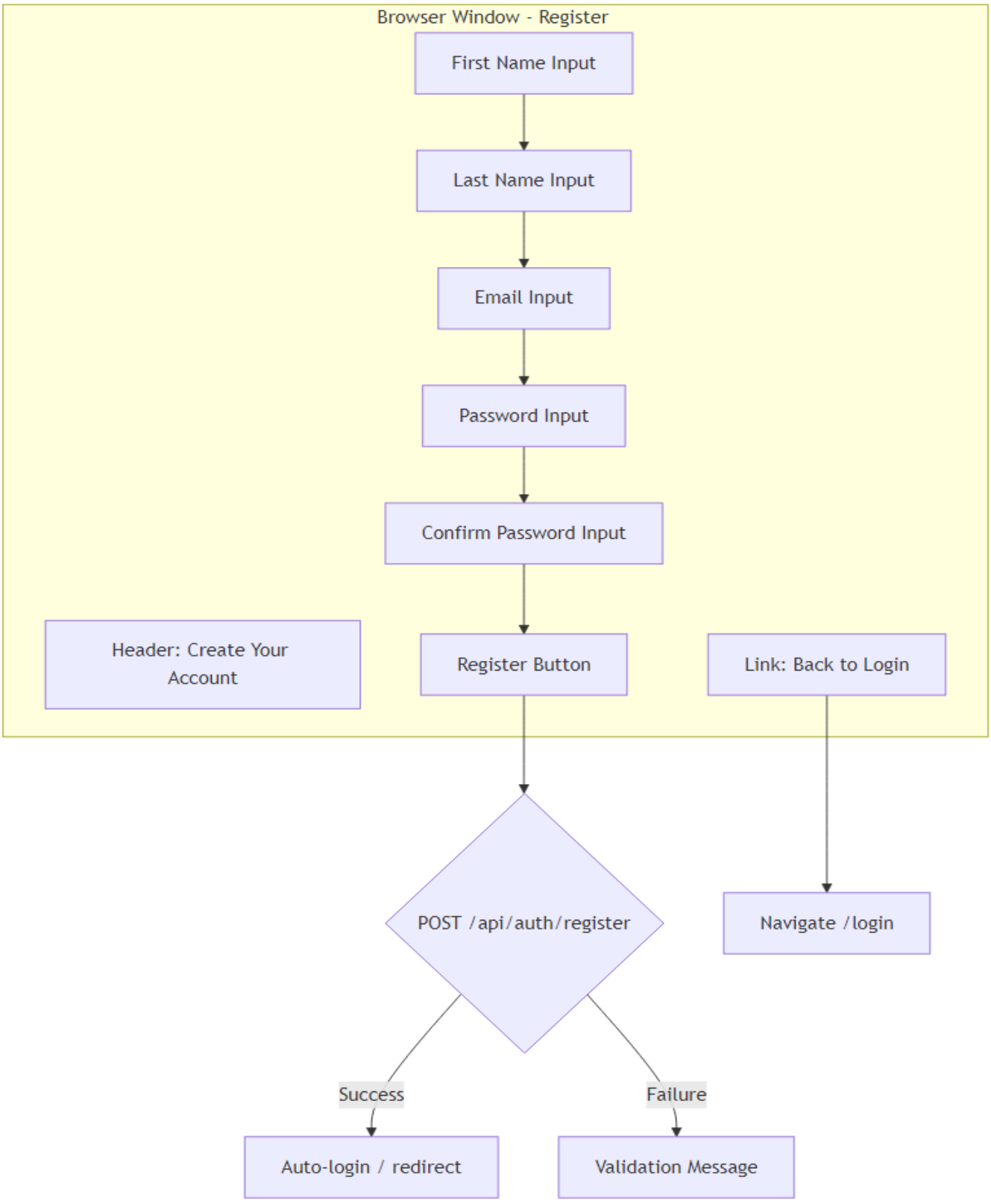


Figure 9.5: Structural UI diagram of registration page components and flow.

User Interface Dashboard Screen

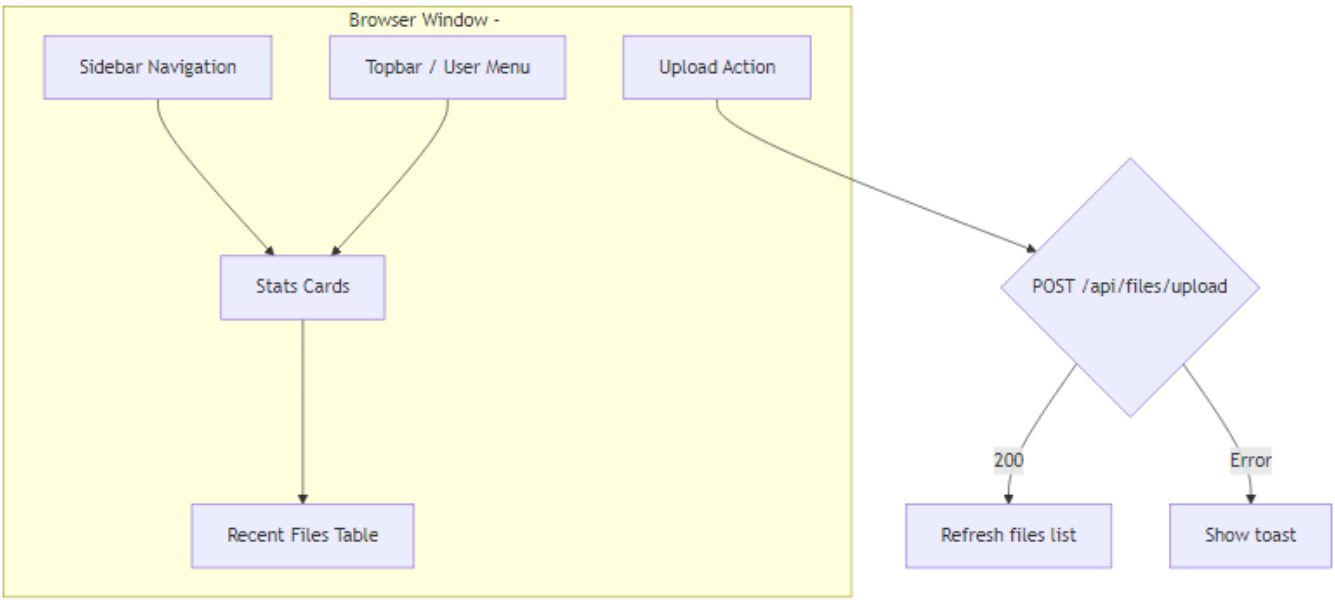


Figure 9.6: Structural UI diagram of dashboard composition and actions.

9.3. BACKEND ENVIRONMENT:

The **Backend Environment** is the "engine room" of the CRM, responsible for processing business logic, enforcing security protocols, and managing data I/O. Built on the **Java ecosystem**, it provides a robust, high-performance foundation capable of handling concurrent file uploads and complex authentication flows.

The server-side environment is designed for scalability and secure data processing.

- **Runtime: Java Development Kit (JDK) 17+**. This version provides modern language features and high performance for the Spring Boot application.
- **Framework: Spring Boot 3.x**. It provides a "standalone" environment with an embedded Tomcat server, removing the need for external web server configuration.
- **Security: Spring Security 6**. Configured to handle stateless JWT authentication and role-based access control (RBAC).
- **ORM: Spring Data JPA (Hibernate)**. This maps the Java domain objects directly to the MySQL database tables, simplifying the database interaction logic

Backend Setup Flow

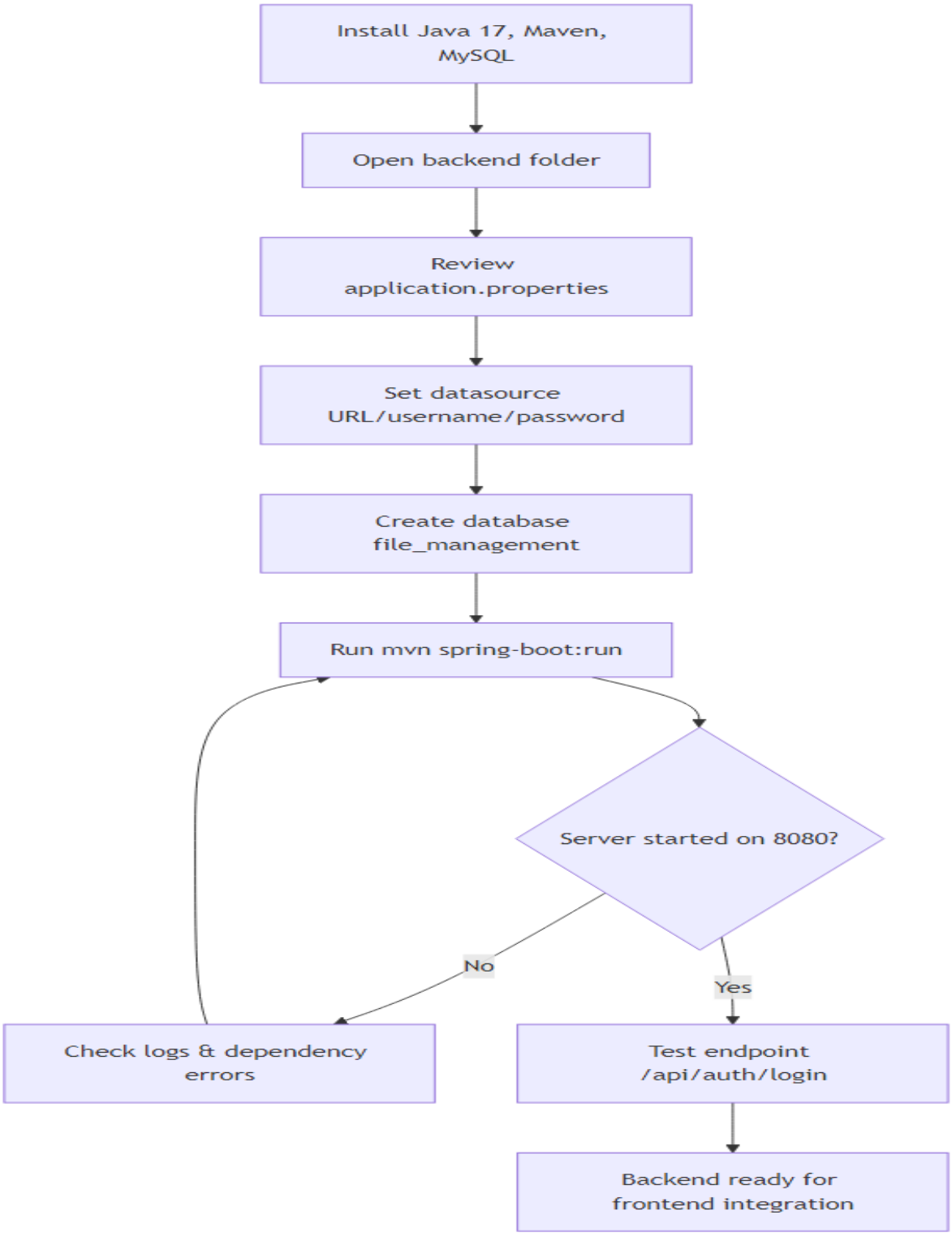


Figure 9.7: Backend setup and runtime verification sequence for local environment

9.4 DATABASE ENVIRONMENT:

The **Database Environment** is the persistence layer of the CRM, responsible for the secure storage and organization of structured data. For this project, a relational model is used to manage user identities and the metadata associated with uploaded files.

9.3.1. Database Management System (DBMS)

- **MySQL 8.0:** Chosen for its reliability, speed, and widespread support in enterprise CRM solutions. It handles the relational data, ensuring that every file upload is linked to a specific user via Foreign Key constraints.
- **ACID Compliance:** The environment ensures Atomicity, Consistency, Isolation, and Durability. This is critical during file uploads; for example, if the database record fails to save, the system can roll back the transaction to prevent data corruption.

9.3.2. Schema Design & Entities

The database is normalized to prevent redundancy and ensure fast query execution.

- **User Table:** Stores unique credentials, hashed passwords, and roles (ADMIN vs CUSTOMER).
- **FileMetadata Table:** Stores descriptors of the binary files stored on the disk. This includes the original filename (for the user's reference) and the UUID-based filename (for system storage).
- **Indexing:** Key columns like username and user_id are indexed to provide near-instant authentication and file retrieval, even as the system scales.

9.3.3. Storage Strategy: The Hybrid Approach

To maintain optimal database performance, the environment follows a **Hybrid Storage Pattern**:

1. **MySQL (Structured Data):** Stores everything that needs to be searched, filtered, or joined (names, dates, sizes).
2. **Server Filesystem (Unstructured Data):** Stores the actual binary bytes of the files. The database merely holds a "pointer" (the UUID filename) to the physical location on the disk.

9.3.4. Security and Maintenance

- **Encryption at Rest:** Sensitive data, specifically user passwords, are never stored in plain text; they are protected using the **BCrypt** algorithm within the database.

9.5 DEPLOYMENT CONFIGURATION:

Local Setup

Frontend

- cd frontend
- pnpm install (or npm install)
- pnpm dev (or npm run dev)

Backend

- cd backend
- mvn spring-boot:run

Configuration Highlights

- server.port=8080
- server.servlet.context-path=/api
- frontend Vite dev port=3000
- frontend API base URL=http://localhost:8080/api

Environment Recommendations

- Use reverse proxy + HTTPS in production.
- Restrict CORS to trusted domains.
- Use managed DB backups and log retention.

9.6. INTEGRATION AND COMMUNICATION:

The Integration and Communication layer defines how the decoupled components of the CRM—the React frontend, the Spring Boot backend, and the MySQL database—interact with each other. It ensures that data flows securely and efficiently across the network using standardized protocols.

9.4.1. Frontend-Backend Communication:

Request Pipeline

1. UI action invokes service method.
2. Service uses shared axios instance.
3. Request interceptor injects Authorization header.
4. Request is sent to http://localhost:8080/api.
5. Spring Security validates token and role.
6. Controller -> Service -> Repository flow executes.
7. JSON (or binary for download) response is returned

Error Handling

- 401/403: frontend clears auth_token and auth_user.
- Browser redirects to /login.

9.7 DEPLOYMENT ENVIRONMENT:

The **Deployment Environment** refers to the final production setup where the CRM application is hosted and made accessible to end-users. Unlike the development environment, this stage focuses on **stability, high availability, and network security.**

Deployment

Layer	Component	Environment Detail
Operating System	Linux (Preferred)	Ubuntu 22.04 LTS
Process Manager	Systemd / PM2	Keeps the Java app running 24/7
API Port	8080 (Mapped to 443)	Secured by SSL/TLS
Storage Path	/var/www/crm/uploads	High-speed SSD storage
CI/CD	GitHub Actions	Automates testing and deployment

Integration Diagram

frontend-backend-integration

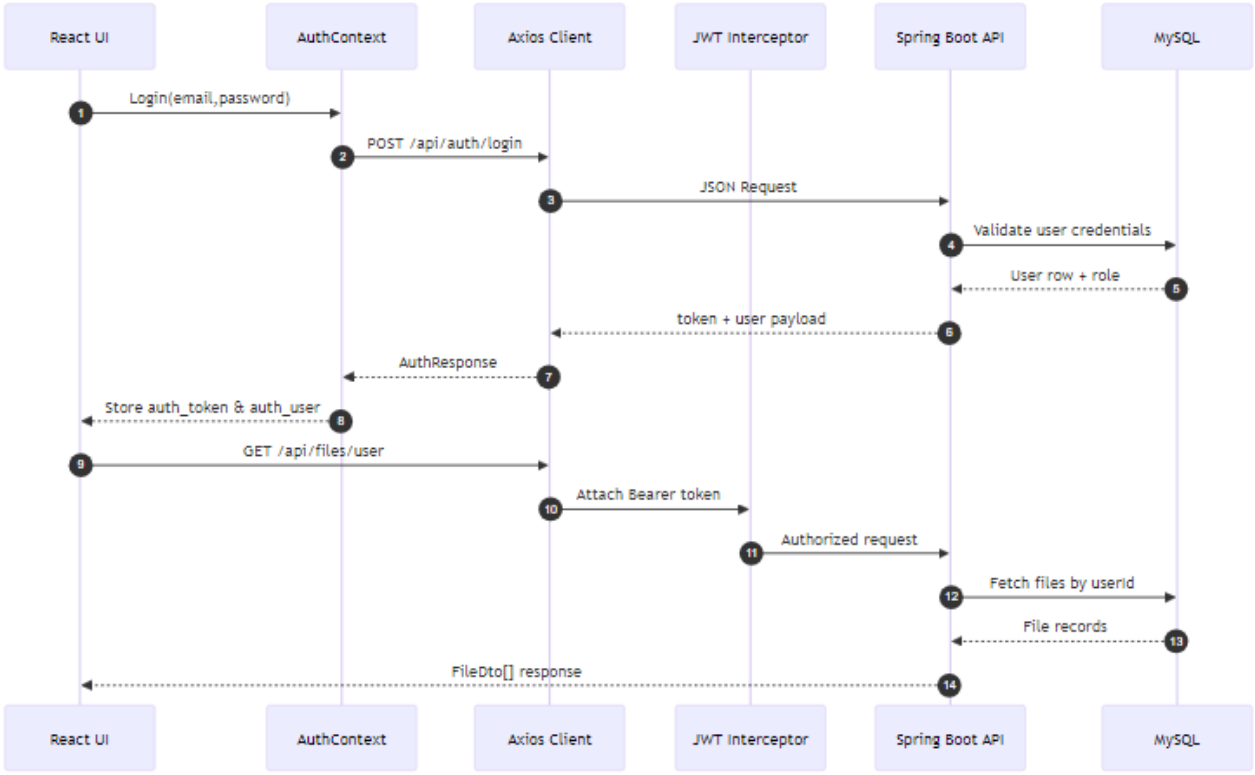


Figure 9.8: Frontend-to-backend request pipeline with interceptor and security layers.

CHAPTER - 10

SOURCE CODE

A) frontend/src/services/api.ts

CODE:

```
import axios from "axios";

// ============================================================
// CONFIGURATION — Change this to your Spring Boot backend URL
// ============================================================
export const API_BASE_URL = "http://localhost:8080/api";

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: { "Content-Type": "application/json" },
});

// Attach JWT token to every request automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("auth_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Handle 401 responses globally
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401 || error.response?.status === 403) {
      localStorage.removeItem("auth_token");
      localStorage.removeItem("auth_user");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);

export default api;
```

B) Frontend/src/contexts/AuthContext.tsx

CODE:

```
import React, { createContext, useContext, useState, useEffect, useCallback } from "react";
import { User, UserRole } from "@/types";
import { authService } from "@/services/authService";

interface AuthContextType {
  user: User | null;
  token: string | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  register: (email: string, password: string, firstName: string, lastName: string) => Promise<void>;
  logout: () => void;
  hasRole: (role: UserRole) => boolean;
  updateUser: (user: User) => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [token, setToken] = useState<string | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    const savedToken = localStorage.getItem("auth_token");
    const savedUser = localStorage.getItem("auth_user");
    if (savedToken && savedUser) {
      try {
        setToken(savedToken);
        setUser(JSON.parse(savedUser));
      } catch {
        localStorage.removeItem("auth_token");
        localStorage.removeItem("auth_user");
      }
    }
    setIsLoading(false);
  }, []);

  const login = useCallback(async (email: string, password: string) => {
    const response = await authService.login({ email, password });
    setToken(response.token);
    setUser(response.user);
    localStorage.setItem("auth_token", response.token);
    localStorage.setItem("auth_user", JSON.stringify(response.user));
  }, []);
```

```
  const register = useCallback(async (email: string, password: string, firstName: string, lastName: string)
=> {
    const response = await authService.register({ email, password, firstName, lastName });
    setToken(response.token);
    setUser(response.user);
    localStorage.setItem("auth_token", response.token);
    localStorage.setItem("auth_user", JSON.stringify(response.user));
  }, []);

  const logout = useCallback(() => {
    authService.logout();
    setToken(null);
    setUser(null);
  }, []);

  const hasRole = useCallback((role: UserRole) => user?.role === role, [user]);

  const updateUser = useCallback((updated: User) => {
    setUser(updated);
    localStorage.setItem("auth_user", JSON.stringify(updated));
  }, []);

  return (
    <AuthContext.Provider value={{ user, token, isAuthenticated: !!token, isLoading, login, register,
logout, hasRole, updateUser }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth must be used within AuthProvider");
  return ctx;
}
```

C) backend/src/main/java/com/example/filemanagement/security/JwtUtil.java

CODE:

```
package com.example.filemanagement.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@Slf4j
@Component
public class JwtUtil {

    @Value("${app.jwtSecret}")
    private String jwtSecret;

    @Value("${app.jwtExpirationMs}")
    private long jwtExpirationMs;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(jwtSecret.getBytes());
    }

    public String generateToken(UserDetails userDetails, String role) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", role);
        return createToken(claims, userDetails.getUsername());
    }

    private String createToken(Map<String, Object> claims, String subject) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + jwtExpirationMs);

        return Jwts.builder()
```

```
            .setClaims(claims)
            .setSubject(subject)
            .setIssuedAt(now)
            .setExpiration(expiryDate)
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {
        return getClaimsFromToken(token).getSubject();
    }

    public String extractRole(String token) {
        return (String) getClaimsFromToken(token).get("role");
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser()
                    .verifyWith(getSigningKey())
                    .build()
                    .parseSignedClaims(token);
            return true;
        } catch (Exception e) {
            log.error("Invalid JWT token: {}", e.getMessage());
            return false;
        }
    }

    public boolean isTokenExpired(String token) {
        return getClaimsFromToken(token).getExpiration().before(new Date());
    }

    private Claims getClaimsFromToken(String token) {
        return Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }
}
```

D) backend/src/main/java/com/example/filemanagement/config/SecurityConfig.java

CODE:

```
package com.example.filemanagement.config;

import com.example.filemanagement.security.JwtAuthenticationFilter;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.builders.AuthenticationManagerBuilder;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {
   @Autowired
   private JwtAuthenticationFilter jwtAuthenticationFilter;
   @Autowired
   private UserDetailsService userDetailsService;
   @Bean
   public PasswordEncoder passwordEncoder() {
      return new BCryptPasswordEncoder();
   }
   @Bean
   public AuthenticationManager authenticationManager(HttpSecurity http) throws Exception {
      AuthenticationManagerBuilder authenticationManagerBuilder =
         http.getSharedObject(AuthenticationManagerBuilder.class);
      authenticationManagerBuilder
            .userDetailsService(userDetailsService)
            .passwordEncoder(passwordEncoder());
      return authenticationManagerBuilder.build();
   }
```

```
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(request -> {
                var corsConfig = new org.springframework.web.cors.CorsConfiguration();
                corsConfig.setAllowedOrigins(java.util.Arrays.asList(
                    "http://localhost:5173",
                    "http://localhost:3000",
                    "http://localhost:8080",
                    "http://localhost:8081",
                    "http://localhost:8082",
                    "http://localhost:8083",
                    "http://localhost:3001"
                ));
                corsConfig.setAllowedMethods(java.util.Arrays.asList("GET", "POST", "PUT", "DELETE",
"OPTIONS", "PATCH"));
                corsConfig.setAllowedHeaders(java.util.Collections.singletonList("*"));
                corsConfig.setAllowCredentials(true);
                corsConfig.setMaxAge(3600L);
                return corsConfig;
            }))
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(authz -> authz
                .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/health").permitAll()
                .requestMatchers(HttpMethod.POST, "/auth/login", "/auth/register").permitAll()
                .requestMatchers(HttpMethod.GET, "/files").hasRole("ADMIN")
                .requestMatchers(HttpMethod.GET, "/admin/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.POST, "/admin/**").hasRole("ADMIN")
                .requestMatchers(HttpMethod.DELETE, "/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

E) backend/src/main/java/com/example/filemanagement/controller/FileController.java

CODE:

```
@PostMapping("/upload")
@GetMapping("/user")
@GetMapping("/{fileId}")
@DeleteMapping("/{fileId}")
@GetMapping("/download/{fileId}")
```

CHAPTER - 11

SYSTEM TESTING

System testing is the phase where the integrated application is evaluated as a whole to ensure it meets the specified requirements. For a CRM file upload system, testing focuses on security, data integrity, and the robustness of the file transfer process.

UNIT TESTING :

Unit testing focuses on the smallest testable parts of the application in isolation. In our CRM, this typically involves testing a single method in a Service class.

- **Mocking Dependencies:** We use **Mockito** to "mock" the database. This allows us to test the FileService logic without actually connecting to MySQL.
- **Key Test Scenarios:**
    - **Filename Sanitization:** Testing if the renameFile() method correctly converts "My Invoice 2024.pdf" into a secure UUID format.
    - **Validation Logic:** Ensuring the isSupportedExtension() method returns true for .pdf and false for .exe.
    - **Password Encryption:** Verifying that the BCrypt encoder produces a hash that is different from the plain-text password.

INTEGRATION TESTING:

Integration testing verifies that different modules of the CRM communicate correctly. This is where we test the "handshakes" between the Frontend, API, and Database.

- **API to Database:** Testing if a POST request to /api/upload actually results in a new row appearing in the File Metadata table in MySQL.
- **Security Integration:** Ensuring the JwtFilter correctly blocks requests to the Admin Module when a Customer level token is provided.
- **Service to Filesystem:** Verifying that when the Upload Service is called, a physical file is actually created in the /uploads directory on the server.

FUNCTIONAL TESTING:

This ensures that the core features work according to the business logic.

- **Authentication Test:** Verifying that users can register, login, and receive a valid JWT. We also test that invalid credentials result in an "Access Denied" message.
- **Upload Verification:** Ensuring that when a file is uploaded, it appears in the user's dashboard and the metadata (size, name) is accurately reflected in the database.
- **Download Test:** Verifying that the "Download" button correctly streams the file from the server back to the local machine without corruption.
- **Admin Access:** Confirming that Admin users can see the global dashboard while Customer users are restricted from it.

SECURITY TESTING :

Since this system handles customer documents, security is the highest priority.

- **JWT Integrity:** Attempting to access the /api/files endpoint without a token or with a tampered token to ensure the server returns a **401 Unauthorized** status.
- **Broken Access Control:** Testing if "Customer A" can download a file belonging to "Customer B" by manually changing the File ID in the URL. The system must block this.
- **File Sanitization:** Attempting to upload a malicious file (e.g., script.exe or malware.sh) to verify that the **File Type Validation** logic rejects it.

USER ACCEPTANCE TESTING:

This is the final check where the system is tested against real-world scenarios. We verify that the **React UI** provides clear feedback, such as:

- Showing a progress bar during uploads.
- Displaying "Success" or "Error" toast notifications.
- Providing a mobile-responsive view for customers accessing the CRM on the go.

AUTOMATED TESTING FLOW:

In a professional environment, these tests are run automatically:

1. **Developer** pushes code to GitHub.
2. **CI/CD Pipeline** (like GitHub Actions) triggers the test suite.
3. **Unit Tests** run first; if they pass, **Integration Tests** begin.
4. **Deployment** only happens if 100% of the tests pass, ensuring that a "buggy" file upload service never reaches the actual customer.

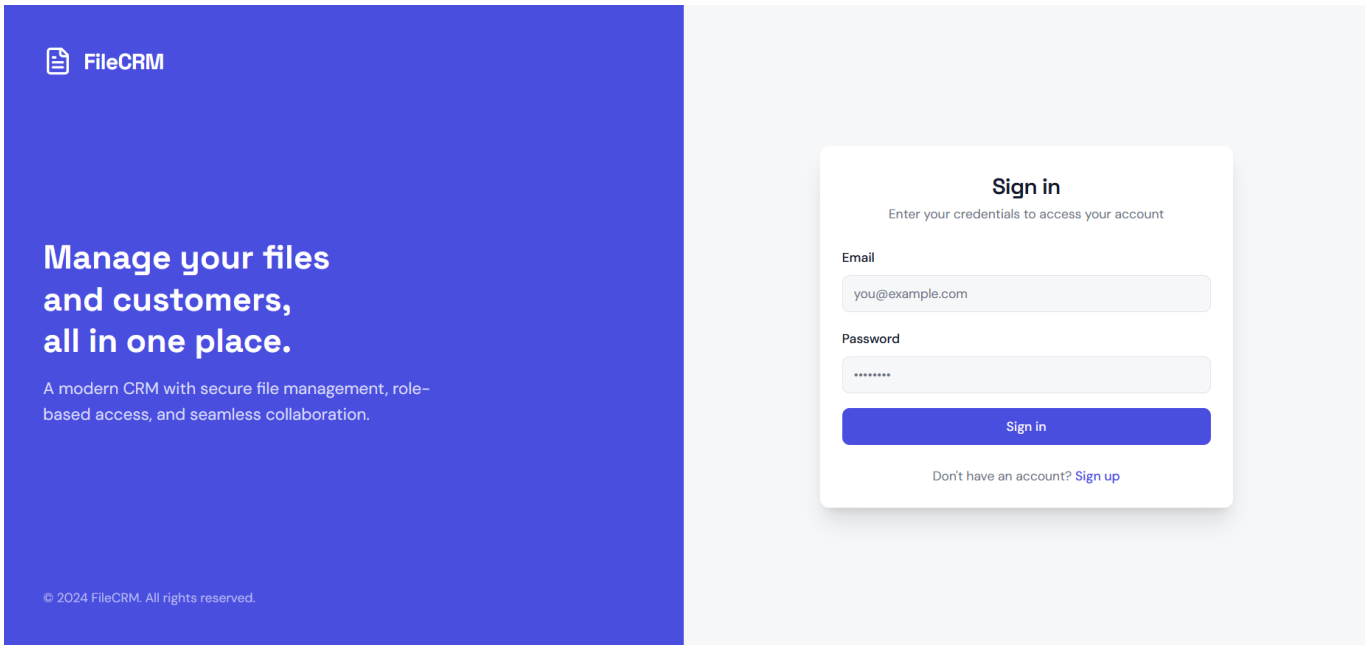
CHAPTER - 12

OUTPUT SCREENS

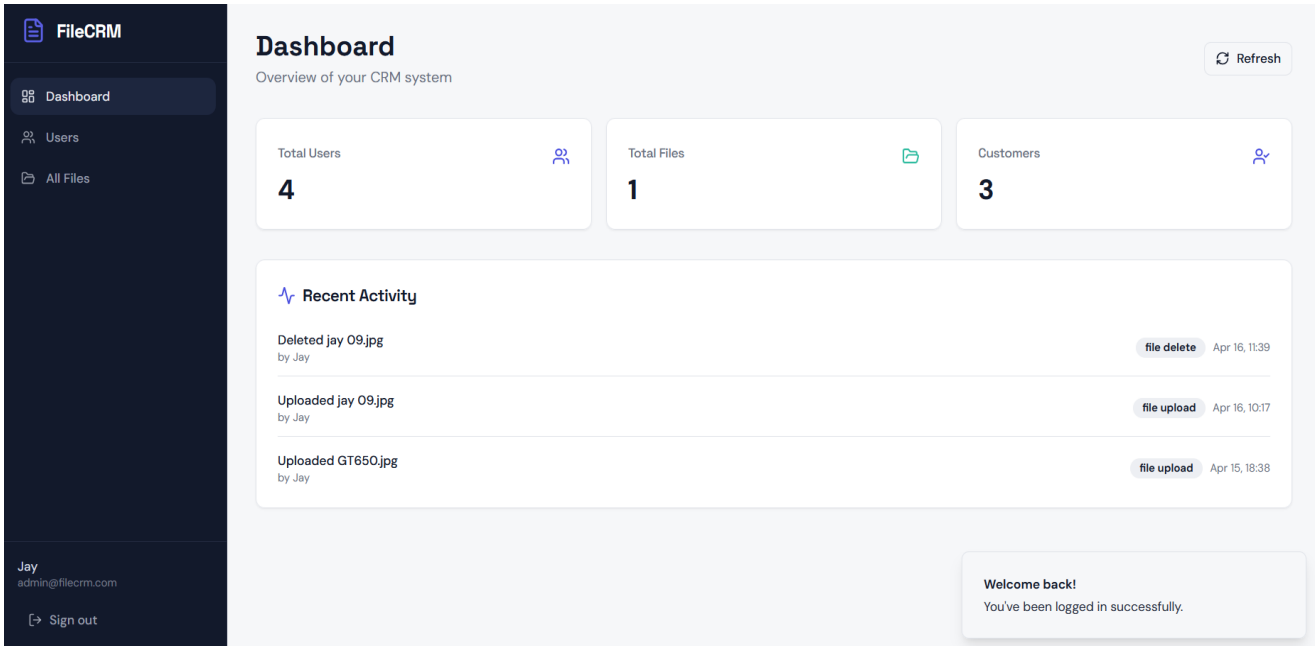
The **Output Screens** represent the final user interface (UI) of the CRM, developed using React and styled with Tailwind CSS. These screens provide a visual representation of how users and admins interact with the file management and authentication systems.

12.1 Admin Interface:

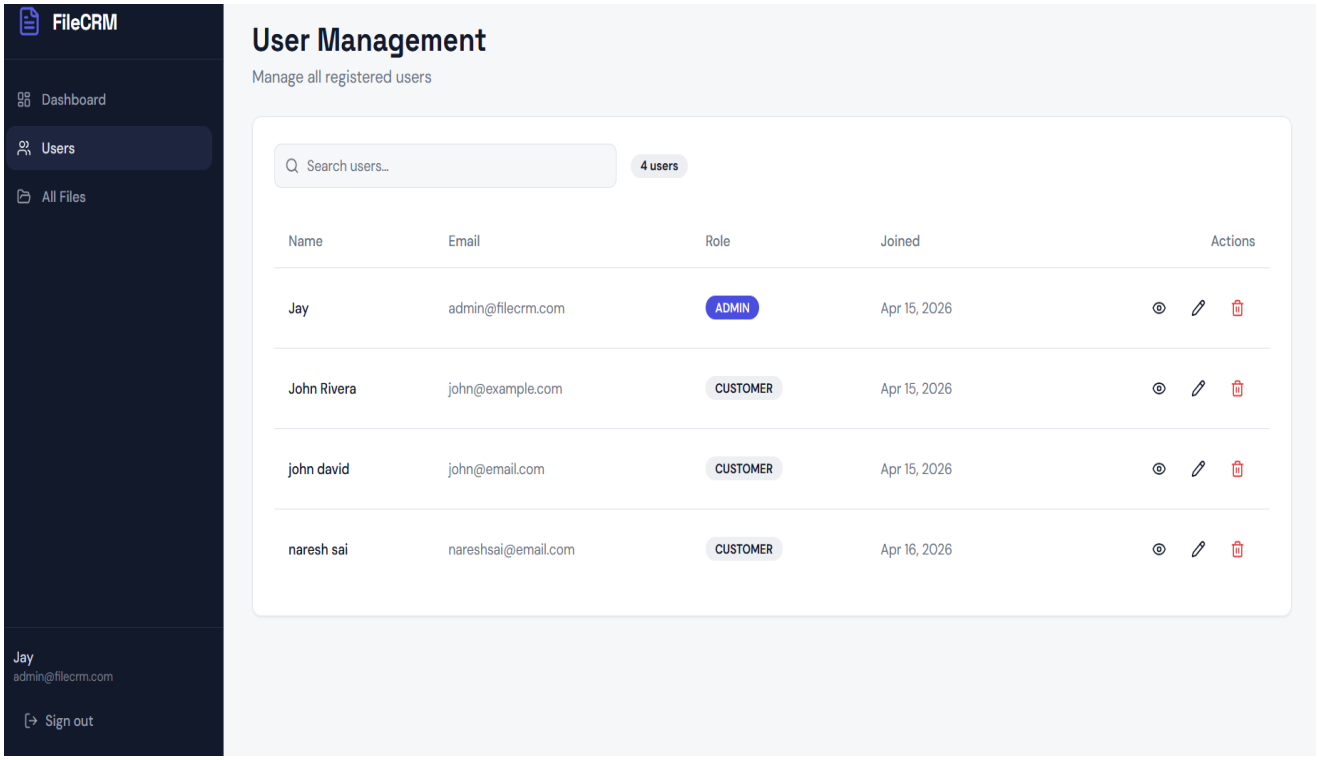
ADMIN SIGN IN SCREEN:



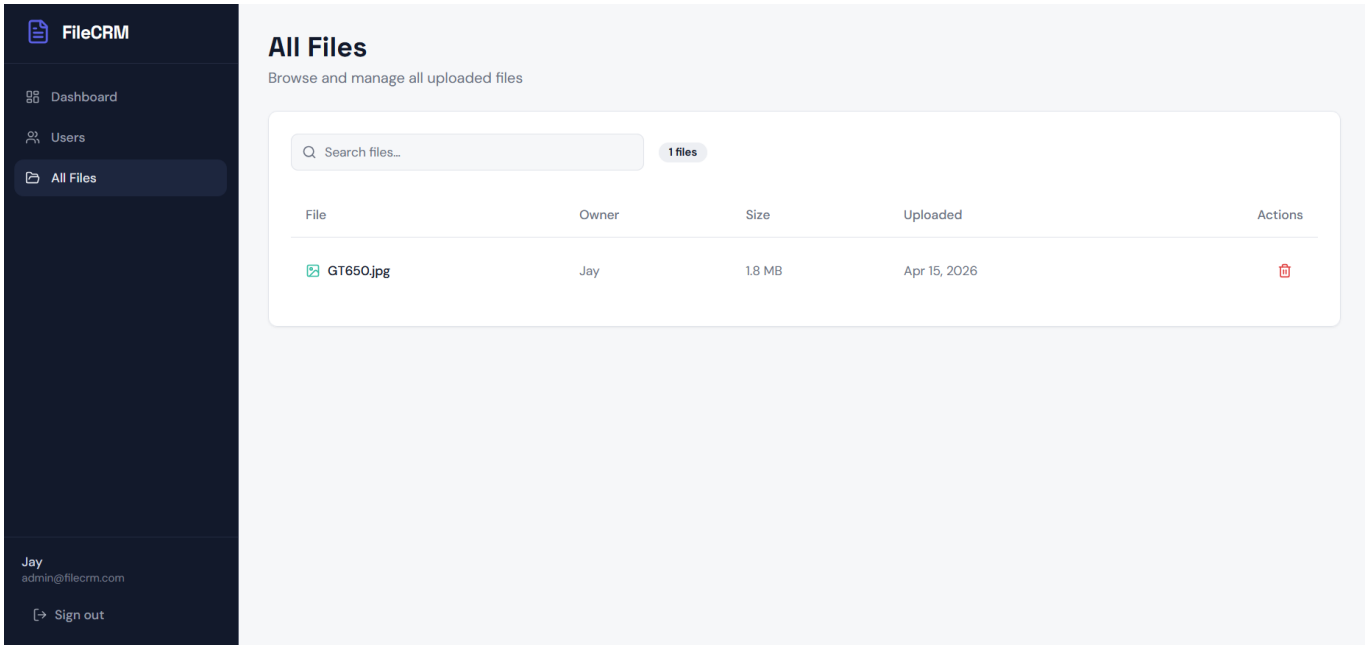
ADMIN DASHBOARD SCREEN:



USER MANAGEMENT SCREEN:

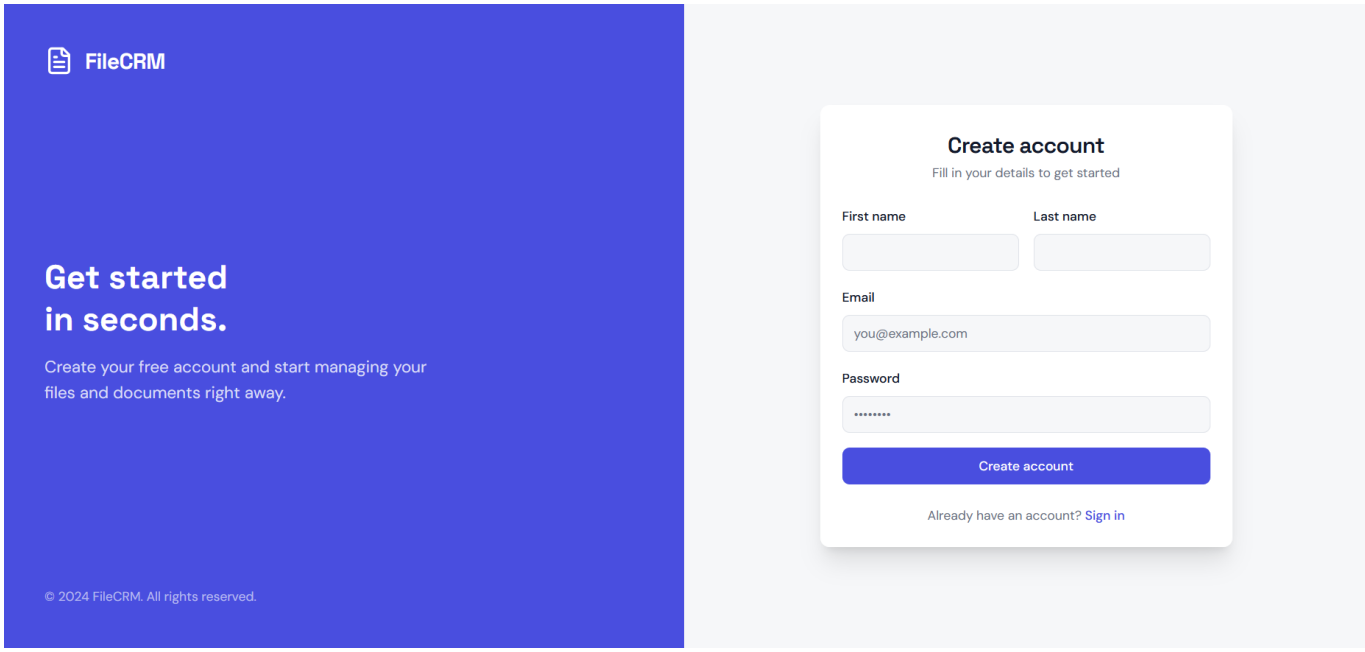


ALL FILES SCREEN:

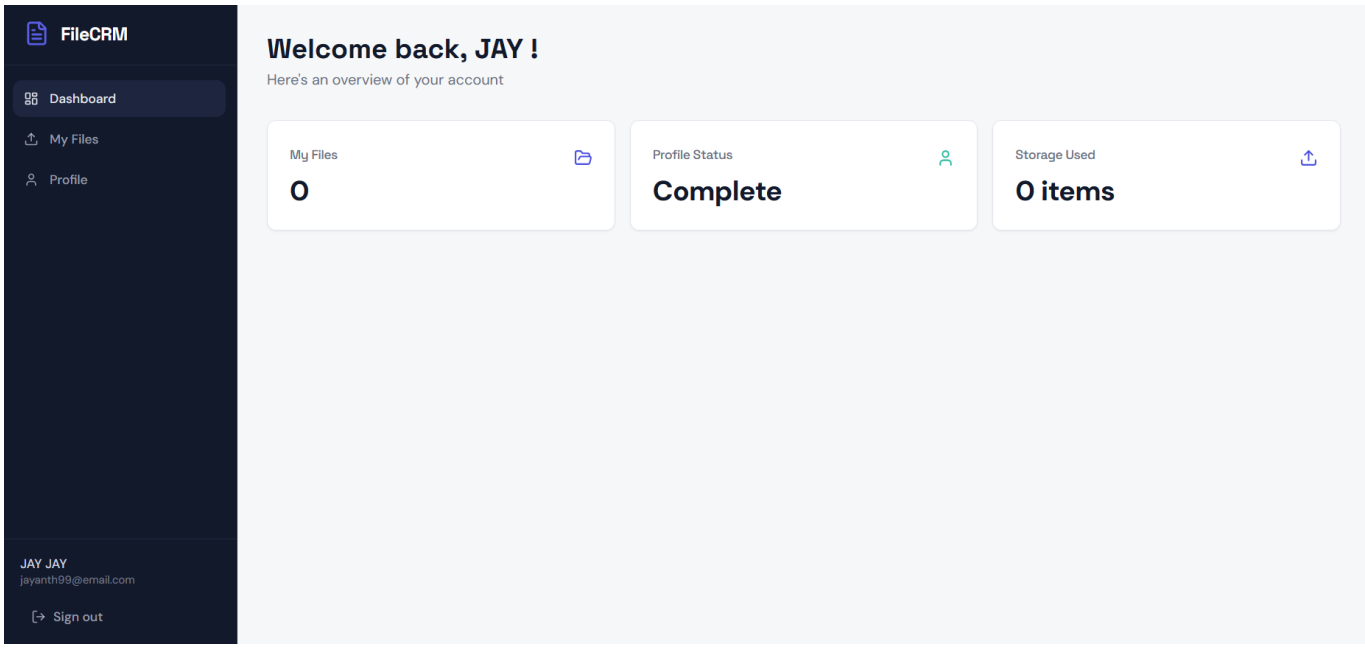


12.2 USER INTERFACE:

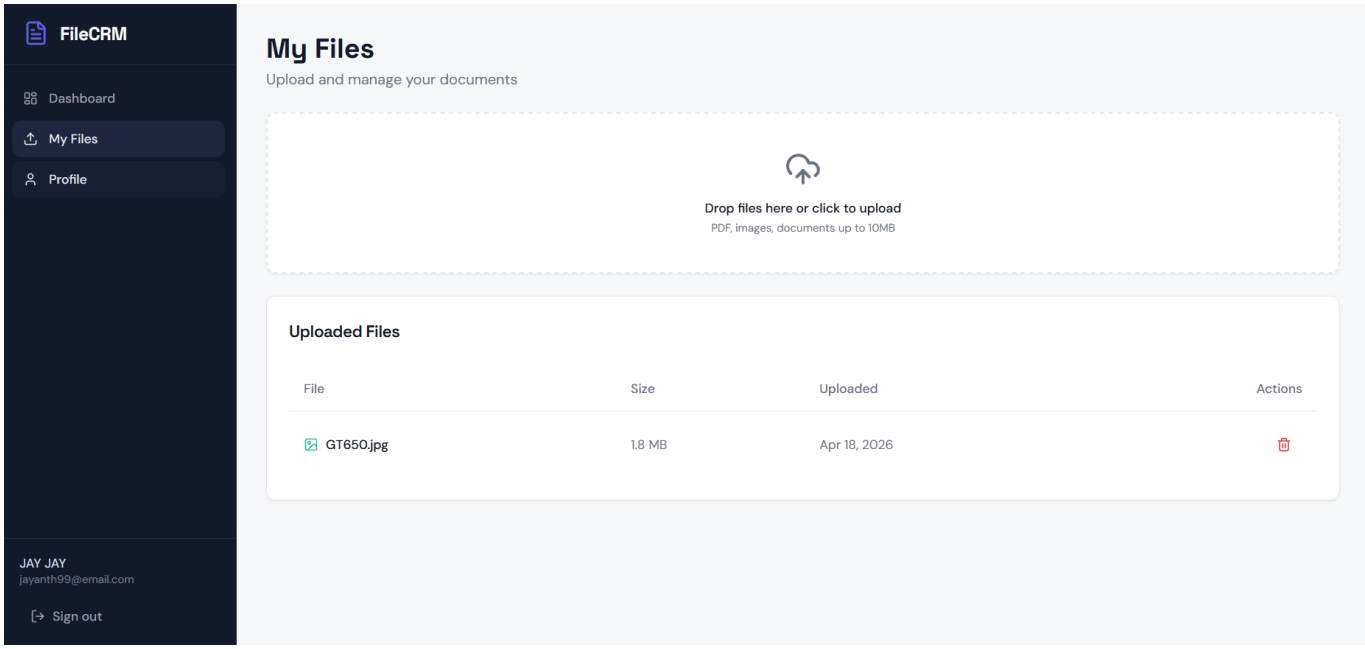
USER SIGN UP SCREEN:



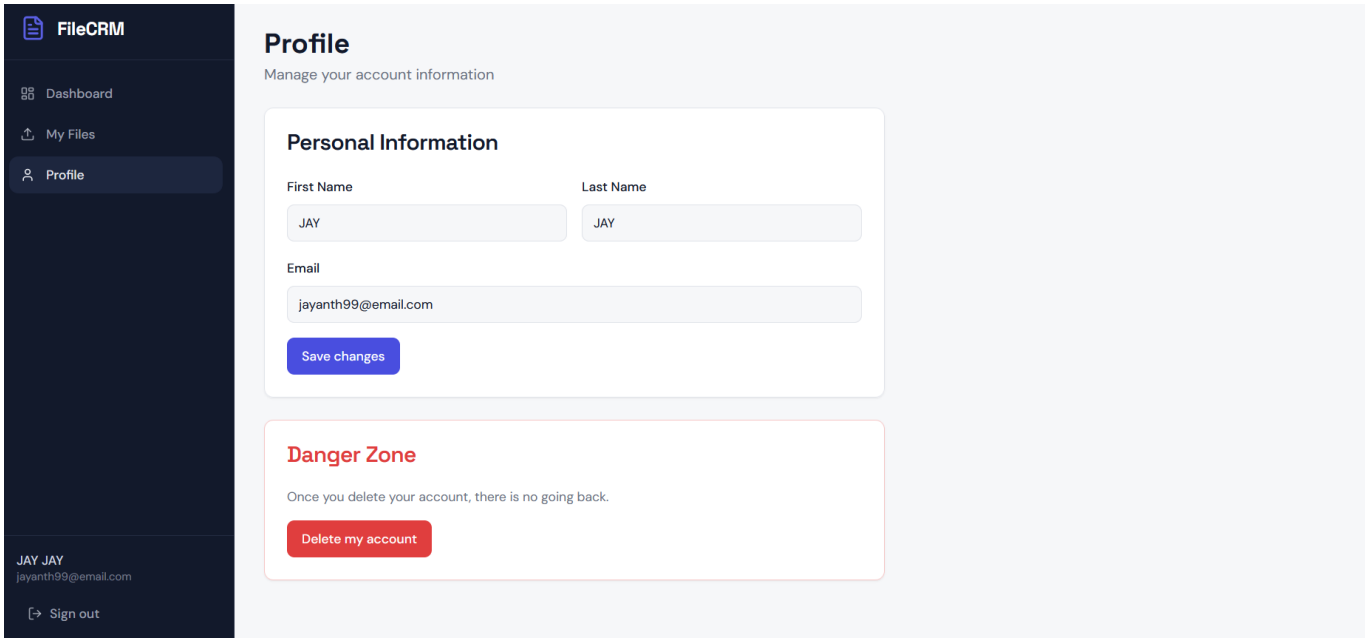
USER DASHBOARD SCREEN:



USER MY FILES SCREEN:



USER PROFILE SCREEN:



CHAPTER - 13
CONCLUSION

The implementation of the File Uploading in CRM system successfully bridges the gap between secure data persistence and intuitive user interaction. By utilizing a decoupled architecture, the project demonstrates how a Spring Boot backend serves as a high-performance engine while a React frontend provides a modern interface. This synergy ensures that the application remains both powerful and user-friendly. The core achievement of this system lies in its ability to handle complex binary data transfers without compromising security. Through the implementation of JWT-based stateless authentication and Bcrypt hashing, the application ensures that sensitive customer information is protected. These protocols prevent unauthorized access and harden the system against common vulnerabilities, maintaining a high standard of data integrity throughout.

A strategic Hybrid Storage Model was adopted to balance system performance with long-term scalability. By mapping file metadata in MySQL while storing physical binaries on the filesystem, the database remains lean and responsive. This approach allows the CRM to manage a massive volume of documents efficiently, ensuring that search queries and retrieval processes remain fast as the system grows.
Beyond technical specifications, this project highlights the necessity of robust error handling within an enterprise environment. The system manages a full lifecycle, including validation against malicious file types and UUID-based renaming to prevent collisions. By enforcing strict role-based access controls, the CRM provides a safe space where administrators and customers can interact without risking any private data leaks.

Summary, this project stands as a production-ready blueprint for modern document management. It offers a stable foundation that is ready for future enhancements, such as cloud migration or AI-driven automation. As businesses continue to move toward digital workflows, the architectural patterns established here will ensure the CRM remains an essential tool for high-performance and secure business operations

CHAPTER - 14

FUTURE ENHANCEMENT

While the current CRM system provides a solid foundation for secure file management and authentication, the modular architecture allows for significant future upgrades. These enhancements focus on scalability, advanced security, and improved user experience.

14.1. Cloud Storage Integration (AWS S3)

Currently, files are stored on the local server filesystem.

- **The Upgrade**: Transitioning to Amazon S3 or Google Cloud Storage.
- **Benefit**: This allows the system to store petabytes of data without worrying about server disk space. It also improves reliability through regional replication and faster content delivery (CDN).

14.2. AI-Powered Document Processing

Integrating Artificial Intelligence can transform the CRM from a passive storage tool into an active assistant.

- **OCR (Optical Character Recognition)**: Automatically read and extract text from uploaded PDFs or images (e.g., scanning an invoice to automatically fill out a price field).
- **Automated Tagging**: Using machine learning to categorize files (e.g., automatically tagging a file as "Contract," "ID Proof," or "Receipt") based on its content.

14.3. Advanced Security Features

To meet enterprise-grade compliance (like GDPR or HIPAA), the following security layers can be added:

- **Multi-Factor Authentication (MFA)**: Requiring a code from an SMS or an app like Google Authenticator during login.
- **File Encryption at Rest**: Encrypting the actual bytes of the file on the disk so that even if the server is compromised, the files remain unreadable without the master key.
- **Antivirus Scanning**: Integrating a service like ClamAV to scan every file for malware immediately upon upload before it is saved to permanent storage.

14.4. Collaborative Tools

Moving beyond simple uploads to allow real-time collaboration between Admins and Customers.

- **File Versioning**: Keeping a history of changes. If a user uploads a new version of "Contract.pdf," the system allows them to "Roll Back" to the previous version.
- **In-Browser Annotations**: Allowing users to highlight text or add comments directly onto a PDF or Image within the CRM dashboard.

14.5. Automated Workflow & Notifications

- **Email/Push Notifications**: Notifying an Admin when a high-priority document is uploaded or alerting a Customer when their document has been approved/reviewed.
- **Expiration Alerts**: Automatically flagging documents that have an expiry date (like insurance policies or IDs) and notifying the user 30 days in advance.

14.6. Administrative Oversight & Analytics

Beyond basic storage, the CRM provides administrators with deep visibility into the system's usage. This ensures that storage resources are used efficiently and security protocols are followed.

- **Storage Quota Management:** Admins can set and monitor storage limits for individual customers (e.g., a 500MB cap for Basic users).
- **Audit Logging:** Every file action (upload, download, deletion) is timestamped and logged with the user's IP address and ID to maintain a clear "paper trail."
- **File Consumption Metrics:** A reporting dashboard visualizes which file types (PDF, Image, Spreadsheet) occupy the most space, helping with capacity planning.

14.7. Data Categorization Schema

To keep the system organized, files are classified into specific categories within the database. This metadata allows for faster searching and filtering.

Data Category	Typical File Types	CRM Use Case
Identity Proofs	.pdf, .jpg, .png	KYC (Know Your Customer) verification.
Legal Contracts	.docx, .pdf	Signed service agreements and NDAs.
Financials	.csv, .xlsx, .pdf	Invoices, receipts, and payment proofs.
Media Assets	.png, .mp4	Brand assets or customer-submitted project photos.